

Entwicklung eines automatisierten Pillenspenders

Bachelor-Arbeit

zur Erlangung des akademischen Grades
Bachelor of
Science in Engineering

Eingereicht am
Bachelorstudiengang Medizintechnik, Linz
Fachhochschule Oberösterreich

von
Roman Iumatov

Linz, am May 26, 2025

Begutachter:
FH-Prof. Dipl.-Ing. Dr. Robert Merwa

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere.

Linz, Austria, May 26, 2025

Roman Iumatov

Abstract

English

The goal of this project is to develop a prototype of an automatic pill dispenser, a device that would assist caretakers with scheduling and dispensing pills for people with limited motor capabilities. This document will cover the details of how it was done, the problems that have arisen as well as solutions to them. We will go step-by-step through each stage of development of such device and highlight challenges and solutions to the task.

Deutsch

Ziel dieses Projekts ist die Entwicklung eines Prototyps eines automatischen Pillenspenders, eines Geräts, das Pflegekräfte bei der Einnahme von Tabletten für Menschen mit eingeschränkten motorischen Fähigkeiten unterstützen soll. In diesem Dokument wird detailliert beschrieben, wie das Projekt durchgeführt wurde, welche Probleme aufgetreten sind und welche Lösungen es dafür gibt. Wir gehen Schritt für Schritt durch jede Phase der Entwicklung eines solchen Geräts und zeigen die Herausforderungen und Lösungen für diese Aufgabe auf.

Executive Summary

The end result of the project is a working prototype of the device. The specifications of the device were defined at the beginning of the project and are as follows:

1. The device contains 21 chambers, 3 for each day of the week.

This requirement is a necessary condition for every pill dispenser, not just the automatic ones, because even the simple dispensers (normally) have a separate chamber for each day.

2. The device contains a pill disposal system.

This is a place where unused pills are stored. This location is also divided into sections so that the caregiver is able to investigate which pills have not been taken and when. Meeting this requirement makes it easier for the caregiver to manage the device, as it is easy for them to analyze unused pills and dispose of the remaining pills.

3. The device has an accompanying app for remote control.

This requirement enables the caregiver to control the device remotely. This is useful for the following reasons:

- (a) The device lacks a control panel on the housing. Therefore, the function cannot be interrupted by accidentally pressing a button.
- (b) With a single Android app, the caregiver may be able to connect to and manage multiple devices remotely. This is useful in hospitals or retirement homes, for example, where there may be several such devices in a confined space.
- (c) Alternatively, for use at home, multiple apps can be connected to a single device (not simultaneously) if there are multiple independent caregivers for a single device.

Contents

List of Abbreviations	7
1 Introduction	8
1.1 Problem Definition and Motivation	8
1.2 Goals and Approach	8
1.3 Structure and Organization of the Work	9
1.4 Tools and Technologies	11
2 Design and development of the Body	14
2.1 Early Ideas	14
2.1.1 Pill Dispense System	14
2.1.2 Pill Disposal System	15
2.2 Selection of the Design	16
2.2.1 Improvements to the Revolver system	16
2.2.2 Redesigning the Revolver system. A new approach	17
2.2.3 The final Redesign. Improving on what's good.	19
3 Construction and 3D-Print of Design	21
3.1 Components of the final design	23
3.1.1 UpperBody	23
3.1.2 UpperMill	24
3.1.3 LowerMill	26
3.1.4 LowerBody	27
3.1.5 Electronics	28
3.1.6 Lower Deck(Electronics compartment)	29
3.1.7 Stand	30
3.2 3D printing specifics	31
4 Microcontroller Programming	32
4.1 Stepper motor controls	32
4.2 Bluetooth Controls	36
4.2.1 Implementation of Bluetooth Protocol	39
5 Development of a Remote-APP	45
5.1 Bluetooth Connection Interface	46
5.2 User Interface	55
5.2.1 MainActivity.kt	56
5.2.2 DevConnViewModel.kt	60
5.2.3 Device Connection	64
5.2.4 Device Configuration	65
5.2.5 Monitoring & Status	67
6 Results Analysis and Discussion	69
6.1 Challenges Encountered	69
6.1.1 Project Management and Execution	69
6.1.2 Technical Complexity and Resource Constraints	70
6.2 Reflection and Lessons Learned	71

6.3	Next steps	71
-----	----------------------	----

List of Figures

1	Early drafts	15
2	First revolver prototype	17
3	2 Variant of the pill dispenser.	18
4	Disposal system of 2nd revolver system	19
5	Vertical Pill Dispenser.	20
6	Final result	20
7	Upper Body	23
8	Upper Mill	24
9	Lower Mill	26
10	Lower Body	27
11	Connection schema of electronic components	28
12	Lower Deck	29
13	Stand	30
14	Issue when slicing	31
15	Stepper Motor Control Logic	33
16	Generic Attribute Profile hierarchy.	37
17	Flowchart of the onWrite method of MyCharacteristicCallbacks class . .	41
18	Model-View-ViewModel Structure	55
19	App Structure diagram	57
20	Device Configuration view.	66

List of Abbreviations

BLE	Bluetooth Low Energy	12
PLA	Polylactic Acid	31
ABS	Acrylonitrile–Butadiene–Styrene	31
OOP	Object-Oriented Programming	13
API	Application Programming Interface	64
GUI	Graphical User Interface	55
VSCode	Visual Studio Code	11
CAD	Computer Assisted Design	11
IDE	Integrated Development Environment	11
UUID	Universally Unique Identifier	38
GATT	Generic Attribute Profile	38
MAC	Media Access Control	36
UI	User Interface	55
MAC	Media Access Control	36
MVVM	Model–view–viewmodel	55
SPI	Serial Peripheral Interface	32
PSRAM	Pseudostatic RAM	32
OSI	Open Systems Interconnection	36
CCCD	Client Characteristic Configuration Descriptor	38
NVS	Non-Volatile Storage	41

1 Introduction

1.1 Problem Definition and Motivation

The main goal of this project is to explore the possibilities of developing such a device that can be mass produced in a decentralized way using conventional tools (3D printing) and at the same time be widely available due to the low manufacturing costs. The question is whether such a device can exist at all. While there are already ready-made solutions on the market, they are not cheap and lack certain features that are present in the prototype in question, be it the lack of remote controls, the lack of flexible scheduling or the lack of a pill disposal system.

1.2 Goals and Approach

The aim of this project is to develop a device that makes it easier for people with motor or mental disabilities to access their medication. This is to be achieved through automation, ease of use and ergonomics. A working prototype of this device will be developed as part of this project. Here are the initial requirements for this project:

1. The device should contain 21 chambers, 3 for each day of the week. This would be the default setting, but the schedule could also be configured to dispense pills either twice or once a day. In this case, the 21-chamber system does not correspond exactly to the 1-week dispensing time. The dispensing information would be available to the caregiver
2. The device should include a pill disposal system. The purpose of this system is to store the unused pills for safety reasons, to prevent accidental overdosing on previously unused pills and to enable monitoring of unused pills. Therefore, this system is also divided into 21 functional chambers.
3. The device will have a companion app. This is an Android-based application that the caregiver can use to configure, monitor and adjust the device.

As this project involves the development of a prototype, activities such as marketing, certification and industrial mass production are not part of the project. User testing is also not part of this project, as it requires the production of several devices, the collection of test subjects (i.e. establishing contacts with hospitals, nursing homes, etc.) and the statistical analysis of this data.

All of these are obviously necessary next steps after the development of the prototype, but for this project they are out of the ordinary.

1.3 Structure and Organization of the Work

This project is divided into several major steps:

1. Design and Development of the Body

This is the first task: developing the basic structure. At the end of this step, a complete device should be brought into shape. We will look at the different options for developing the structure, discuss their pros and cons and choose the most suitable one for the next step.

2. Construction and 3D-Printing of the Design

This is the step where our design takes on a physical form. This is the prototyping step where we iteratively develop the device to make sure the design works. This is also the step where the material and mechanical properties of our device are our biggest concern. At the end of this step, the device is functional but not yet configurable. This means that it will be able to rotate one chamber at a time.

3. Development of the Android APP

This step is similar to steps 1 and 2, but for an Android app. We design and develop the template for the app, which can then be filled with functions. In this step, we will also determine which functions the microcontroller on our device must output to the app. At the end of this step, we would have a working device and an app, but they are not yet connected, which brings us to the next step.

4. Development of the interface between the device and the app

This is the final step where everything comes together. The device will communicate with the Android app. It will be able to send (usage statistics, current time, time until next delivery, etc.) and receive (configuration settings, forced delivery command, etc.) information.

The table contains more detailed and precise steps to be taken during this project. Please note that it has been expanded and therefore includes more steps. This means that several steps from the table below are combined into one step from the list above. More specifically, steps 1.0 and 2.0 belong to the Design and development of the Body section, and steps 3.0 and 4.0 belong to the Construction and 3D-Print of Design section.

Work Package	Input	Activity	Goal
1.0 Initialization	Definition of the precise project requirements, identification of the target audience, execution of the bureaucratic process, definition of the project scope.	Communication with stakeholders	Clearly formulated objective, structured development plan.
2.0 Physical Requirements	Requirements for the device's physical properties: materials, mechanisms, size, robustness requirements, ergonomics.	Analysis of the target audience and communication with stakeholders.	Definition of the boundaries for the proposed designs.
3.0 3D Printing and Construction of a Functional Prototype	Requirements for the materials used, 3D models of the prototype.	Adapting the models for 3D printing, printing the prototypes.	Physically existing device with all functional features.
4.0 Implementation of the Delivery System	Constraints from the previous step, ideas for implementation, feasibility of different approaches.	Defining and finalizing a 3D model of the delivery system, 3D design, optional 3D printing of a prototype.	Functional delivery mechanism.
5.0 Implementation of the Control System	Requirements for ergonomics and functionality.	Selecting the microcontroller, connecting the microcontroller to the delivery system, creating skeleton functions for the required device features.	Delivery system and control system are interconnected and can be programmed, making the delivery system controllable.
6.0 Programming the Remote App	Requirements for the remote app's functionality.	Programming the app (in a high-level programming language).	Application capable of controlling the device over a wireless network.
7.0 Programming the Control System	Skeleton functions implemented in the previous steps.	Programming the microcontroller.	The device functions correctly.

Table 1: Project planning table

1.4 Tools and Technologies

For each step there is a different toolset that will be covered here and then mentioned in the chapter of the particular step if necessity arises. Here are the tools that have been used:

1. Organisation and Project Management

- **GitHub**

Is a version control tool that has been used for bug-fixing, change tracking and logging of the work done. It is used extensively for Frontend and Backend development steps. It is also used to synchronize work from different machines. The project is organized in such a way, that source code for Microcontroller, Android App are all within the same repository.

- **AI Tools**

These tools are used mostly for troubleshooting and debugging. The process is usually this: When there is an issue that is hard to resolve, the description of the problem and all relevant information is then fed into all 3 of the used Language Models, the final solution is typically derived from multiple suggestions, rather than adopted verbatim from a single model. This is done so because these models can be confidently incorrect at times, which, if their advice is taken without understanding, can lead to a deeper level of confusion. It will also be mentioned in the work whenever AI was used to generate solution. All AI-generated code was subsequently reviewed and adapted by the author; responsibility for final content remains with the author

These three are used together in a way described earlier:

- **OpenAI ChatGPT o3**

- **Google Gemini 2.5 Pro + AI Studio**

- **Deepseek R1**

GitHub Copilot is integrated within Visual Studio Code (VSCode) as a plugin. It is sparingly used whenever there are smaller issues, mostly syntax errors to be fixed, as unlike the three models mentioned above it is not a thinking model.

Gemini is integrated into Android Studio. While Android Studio is a powerful, modern Integrated Development Environment (IDE) on its own, offering many suggestions and analytics, it has also an AI-Model built in (however version is not known, neither is it disclosed anywhere in the Android Studio documentation). It was used sparingly when IDE suggestions didn't help.

2. 3D Model Prototyping and Design

- **Fusion 360** was the primary Computer Assisted Design (CAD) software used for all stages of 3D modeling. It is the main and only tool used for Prototyping Earlier Sketches and creating 3D models of the finalized prototype. Unlike other 3D-Modeling software that relies primarily on meshes (Blender, 3DMax), Fusion 360 Uses parametric design which is more precise, with further slicing for 3D-Printing (further facilitated by built-in export to .stl feature) and contains a wide toolset of analytical features (such as Interference, Center of Mass and Section Analysis which were used extensively). Its parametric nature also

allows easy adjustment of dimensions of components whenever an overlap between them happens.

- **PrusaSlicer** This software is used as an intermediary step between a 3D-model and 3D-printer. Since printers require a set of commands to execute and a .stl is a geometry data, there needs to be an interpreter software which would translate this data into an instruction set (G-Code, although PrusaSlicer has a very convenient feature of transferring this code directly to the printer if computer with the slicer and 3D-Printer are connected to the internet). All of the prints have been done with the default built in template for Prusa MK4S called **0.20mm STRUCTURAL**. Some models required support material to be added and since original template doesn't have them enabled (either for autogeneration or for printing) this template would then be adjusted to include printing supports.

3. Frontend Development

- **Android Development Studio + Kotlin** This combination is a industry standard in Android development. Google has established that Kotlin is preferred language for developing an Android apps [1].

Android Development Studio IDE is a joint project of Google and JetBrains, an innovative IDE development company. it has many powerful features such as Previews, Device Emulators, Layout Editor and Gradle Build System. These features have alleviated the development process and debugging process.

Kotlin was chosen because an alternative would be Java, which is not personally preferred development language. Besides being industry standard, Kotlin has very easy-to-understand syntax, *Null-Safety* (a feature of the language, which prevents certain variables to have a null values i.e. value, the pointer of which doesn't have an address), that prevents very common Null-pointer exception runtime errors and also google's official trainings and tutorials which were essential in learning the language.

- **nRF Connect** is an Android app that allows troubleshooting Bluetooth Low Energy (BLE) connection issues. it displays important information about BLE stack of the connected device, which helps finding "lost" characteristics.

4. Backend Development

- **VSCode + Platform.io + C++** Similar to the Frontend development step, this is also quite common choice for programming. VSCode offers a wide variety of plugins, due to being open source, and Platform.io is one of them.

VSCode is chosen mostly because of ease of use and flexibility. It is Open-Source, therefore there has been a huge community effort to develop plugins and also one can adjust almost every component of the editor as one sees fit, for example by creating and running custom scripts.

Platform.io VSCode plugin has many important features, such as Microcontroller templates, built-in C++ and Arduino libraries, Device monitoring (data from connected device can be logged into terminal) and Upload of compiled code. As mentioned before, it is a standard for Arduino software development

and enjoys a wide support from the community. This is important not only for the development of a device, but also for maintenance of its code in the future, should a project be further expanded.

C++ Was chosen because of efficiency, due to being one of the lower-level programming languages, close to hardware, while also being able to abstract away, reducing the need to manually flip bits on the hardware. Furthermore, unlike C, another commonly used language for hardware programming, C++ supports Object-Oriented Programming (OOP), whose features (Incapsulation, Polymorphism, Inheritance) offer aid in programming a device.

2 Design and development of the Body

The development of a device usually involves several steps, starting with the design phase. In this phase, a decision must first be made as to how the device should look in view of the requirements that are set. The requirement to have 21 chambers is of particular interest in this context, as it significantly restricts the design options. However, this restriction should be viewed positively, as the requirements for the first steps seem to be too low. There are no requirements regarding the physical characteristics of the device, such as size or weight. This opens up a remarkably wide scope for design with regard to the external appearance of the device.

Based only on the information we have at hand, there are already some initial ideas about what the device might look like, each with its own advantages and disadvantages. Here is a brief overview of the possible designs we have come up with.

2.1 Early Ideas

2.1.1 Pill Dispense System

When thinking about how the pills should be dispensed, a number of ideas inspired by the environment emerged. Below you will find brief descriptions of the systems and also an image 1 of what they would look like. Please note that we are not going into too much detail here as they were the result of a brainstorming session and after all that, only one idea would be developed moving forward, albeit with certain inspirations from ideas that were dismissed.

- The **Revolver** system was the simplest, and it didn't take long to come up with this idea, because such devices had already been developed[2]. Its advantages are the simplicity of the mechanism, the robustness and the low price, while a disadvantage is its size and it requires a motor to drive a fairly large wheel.
- The second design is that of the **Carousel**. The main idea is that it is much smaller and uses space more efficiently than the revolver system. The smaller wheels would rotate at a lower speed than the main arm that holds them, as they are attached to the main axle with a reduction gear and a belt. Although it is slightly more space efficient than a revolver system, it is much more complex and still requires a motor to control the entire structure.
- The third design is that of the **Conveyor Belt**. On the image 1 you can see that it is round, but it could theoretically also be designed elliptically or with other, more complex curves. The principle is as follows: An output chamber is attached to the belt, which moves a series of these chambers along a specific path. It is similar to the revolver system, but the chambers would be removable and the conveyor belt would allow some flexibility in the design of the device. The main disadvantages are the need for a motor, the need for a chamber management system and the overall complexity of the device.
- The fourth design is inspired by **Vending Machines**. It would have 21 chambers, each with an electromagnet attached. Each time a pill is to be dispensed, the electromagnet is activated, causing the chamber to move and the pills to fall onto the dispensing area. In this design, the stepper motor becomes redundant, but

simplicity is lost as a logical circuit is required to connect 21 chambers in parallel. Theoretically, a parallel circuit would be advantageous, but this is not the case for us, as we cannot leverage this advantage sensibly.

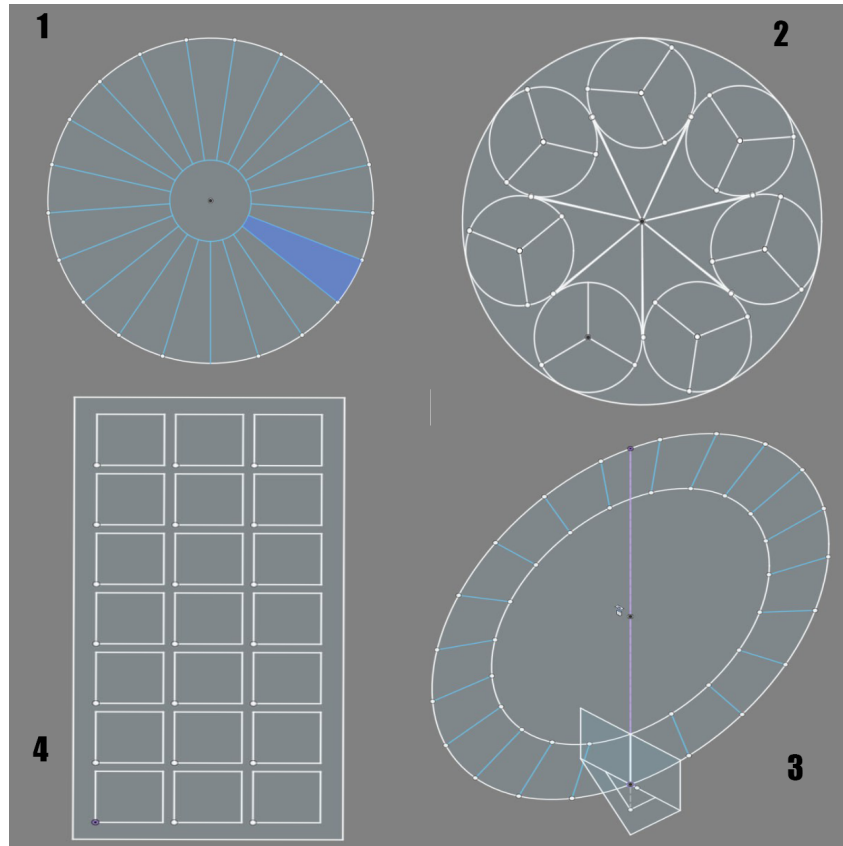


Figure 1: Initial ideas on how the main structure (dispenser) of the device should look. 1. Revolver system. 2. carousel system. 3. conveyor system (running sushi). 4 Vending machine system

2.1.2 Pill Disposal System

The idea for the Disposal system at the time was that it would mirror the dispensing system, either by being positioned directly underneath it or in such a way that the tablets would first fall onto a ramp, which after a certain time would be transported into the return mechanism by another action. It is not difficult to see that the choice of disposal system is highly dependent on the dispensing system we develop, so a more detailed description will be given later when we look in more detail at the chosen dispense system.

2.2 Selection of the Design

Of all the proposed designs, the revolver system seemed to be the most sensible choice, but it required some improvements in the design. A brief market research (e.g. [2] [3]) showed that there is not really such a system that would meet the above requirements of having a built-in disposal mechanism. For this reason, the design would have to be improved.

2.2.1 Improvements to the Revolver system

Now that we have chosen the main mechanism, we need to work out the details. The design philosophy was to make it as simple as possible, with as few moving details as possible. The first ideas were not very successful in this respect. In the picture 2 you can see what the earlier ideas would look like. In such a design, each chamber would have its own door, under that door would be a notch for the lever to fall into to open it. The pills would then fall into the disposal tray. After a certain amount of time (15 minutes), the S-shaped arm on the disposal platform would rotate and drop the pills further down the ramp into the disposal storage. This solution had numerous disadvantages:

1. Too many moving parts

Each chamber has 21 doors. The doors need a lever to open (22), this lever needs to be translationally engaged and disengaged in the notch (23), the revolver mechanism needs to rotate (24), the disposal arm also needs to rotate (25, but the idea was to synchronize it with the main axis using belts), the disposal chambers would also need to rotate (26). Although this was a solution, it was not optimal given the number of moving parts. Although the dispensing chambers, the disposal chambers and the disposal arm would rotate around the same axis, the rotation and translation of the lever for an opening would require a complex mechanism that would further increase the effort.

2. Requires a large dispenser bowl

The disposal mechanism itself is already quite large (a diameter of 300 mm was measured for this prototype). If another horizontal surface were added next to this surface, even if it were only half the size (a reduction in size would make it difficult for people with motor disorders to remove the pills safely), the entire system would become extremely large.

3. Contains several small objects

This is not an obvious disadvantage, but each trapdoor must rotate around a specific axis. This axis must consist of either a small metal rod inserted into the openings of the door or 3D printed teeth on the design of the door itself. The first solution requires a daring and precise assembly of 21 doors, the second requires a high level of 3D printing precision that cannot be achieved by conventional means.

Due to these disadvantages, the system was discarded and never further improved. However, one important lesson was learned: if we want to have the simplest design, we have to utilize the power of gravity.

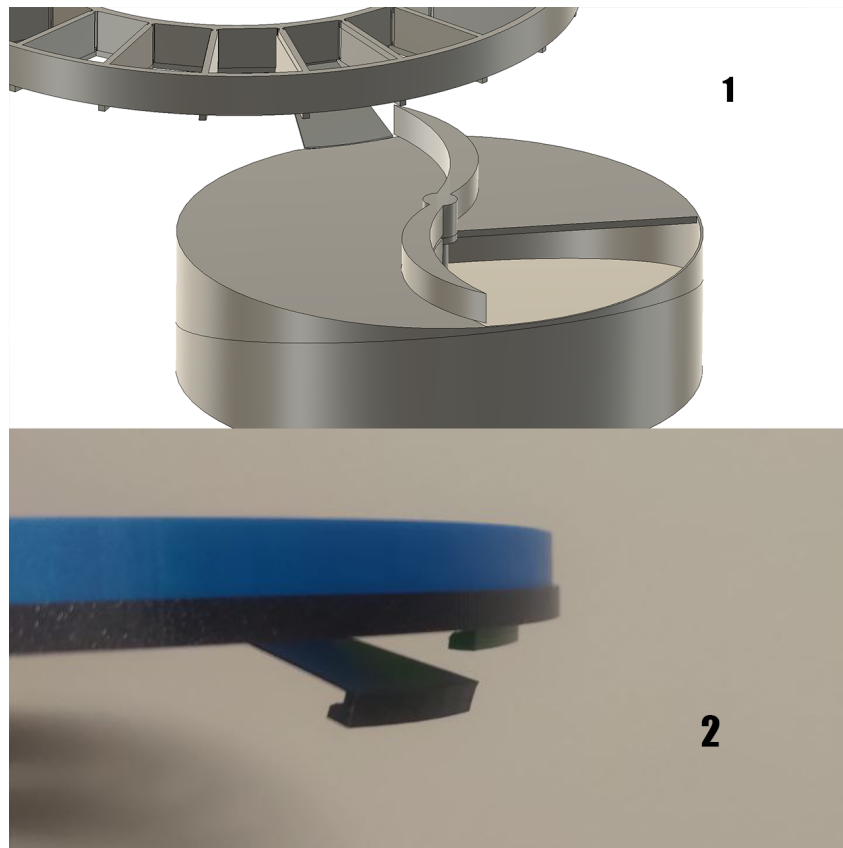


Figure 2: First revolver prototype. Here you can see that initial idea was to have a trapdoor in each chamber which will release the pills onto the disposal dish by opening it. (1) Is a designed prototype, (2) is a photo of 3D-printed prototype

2.2.2 Redesigning the Revolver system. A new approach

Having realized that we need to get rid of the extra space that the separate body at the side of the main body takes, the new approach was required. This time, the disposal chambers would be directly underneath the dispensing chambers and we would use the power of gravity to move from one chamber to another, as you can see in the image 3. This would drastically reduce the amount of space the pill dispenser takes, significantly reduce complexity, setting all the rotating objects onto a single axis.

Because this system doesn't have a separate mechanism for removing unused pills, another new method would have to be developed for this too. In the image 4 you can see how the chambers of the disposal system look like. The cover of the disposal system has an opening (part 2 of the same image) that is shut when the pills have to be taken. When the upper mechanism rotates to dispose new pills, the lower mechanism rotates to send pills into disposal chamber simultaneously. Ridges are added to the cover of disposal system to prevent pills from accidentally rolling away.

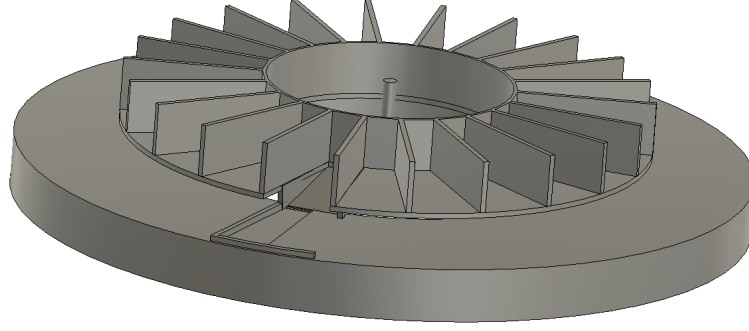


Figure 3: Second variant of the pill dispenser, with unevenly sized chambers

However, there are still ways to improve this approach. While much better than the previous one, this one also have some disadvantages:

1. **Chambers of the upper mechanism are smaller.** The chambers of pill storage are significantly smaller, as clearly seen on image 3. We can calculate chamber volume using the following formulas:

$$A = \pi r_1^2 - \pi r_0^2,$$

$$V = A \cdot h$$

For the upper chamber: $r_1 = 100 \text{ mm}$, $r_0 = 47.5 \text{ mm}$, $h = 18 \text{ mm}$, the result is:

$$V = 437.90 \text{ cm}^3$$

For the lower chamber: $r_1 = 135 \text{ mm}$, $r_0 = 80 \text{ mm}$, $h = 18 \text{ mm}$, the result is:

$$V = 668.69 \text{ cm}^3$$

As we can see, the lower chambers are almost 1.5 times bigger than the upper ones, which is a waste that we might want to get rid of.

2. **Synchronous movement of both chambers** This approach makes future development less flexible. independence of the chambers is a degree of freedom that we might want to keep somehow. The biggest problem with current approach is that it might lead to situations where some pills might fall too fast from the top to fall directly into an opening in time.
3. **Reliance on a ramp for pills to fall down.** This can introduce an undesired behavior where some sticky pills might stick to the ramp and not fall down. This can lead to very dangerous scenario where a pill, not properly dispensed in time, would fall down with the next batch which would then lead to overdose.

As mentioned earlier, this design is already what we would like to see as a final result, but we can improve it further, by redesigning it a bit and adresssing the disadvantages we mentioned above.

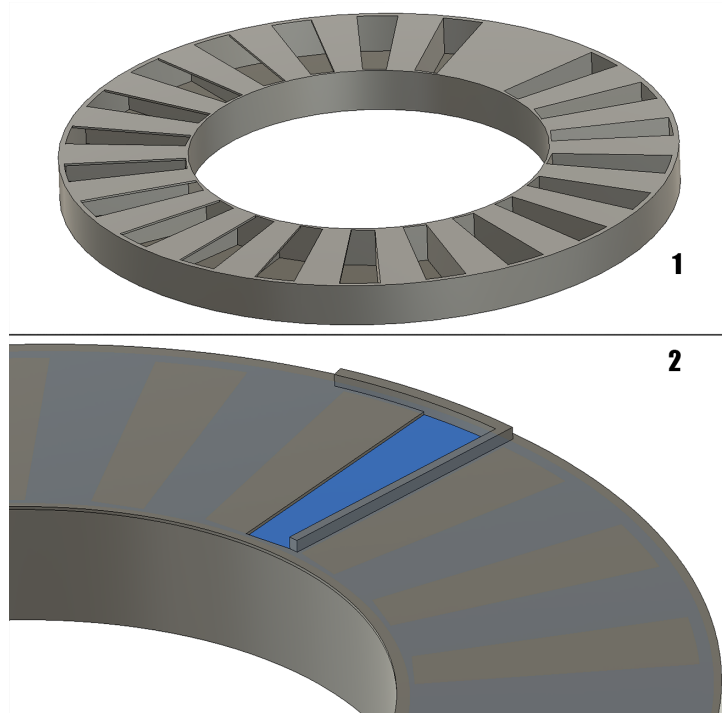


Figure 4: Decomposed Disposal system of 2nd revolver system. (1) Shows the Lower Mill without cover. the whole construction would be rotating, covered by a ring with a cutout, seen on (2)

2.2.3 The final Redesign. Improving on what's good.

A few thoughts come to mind as a potential approach to the redesign. Using gravity is obviously good, we need to continue doing, however we might change how we use it. Having 2 systems one above another is also a great idea, what if we can make the chambers the same size? We could reposition the holes. Regarding synchronous movement, we have 2 types of direction of pill movement: From dispense to patient (intended behavior) and from dispense to disposal (backup behavior). The intended behavior implies human interaction, we can make some simple movement to deliver pills that would remove the need for synchronicity from our system. All these ideas are implemented in the final design, however it is worth mentioning that there has also been the intermediary step, where the dispense mechanism was rotated 90 degrees (see image 5), so that it is completely vertical. It is worth giving a short overview of why this idea wasn't chosen as final:

- **Too tall** the wheel with diameter of 100mm would be vertically positioned which takes a lot of space.
- **Requires more torque** A motor would have to directly resist the force of gravity to lift the pills up a mill
- **Doesn't fix synchronicity** The mechanisms are still locked together, this issue remains unresolved.

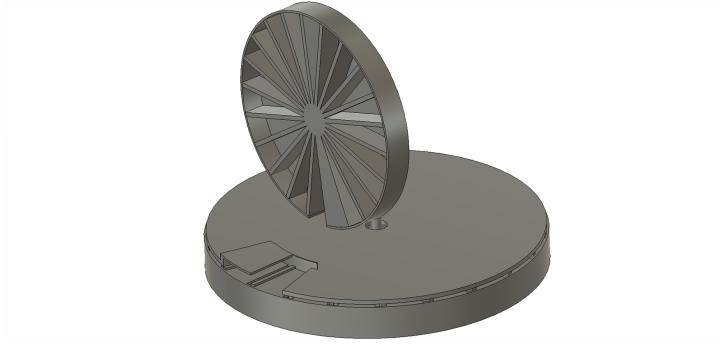


Figure 5: Pill dispenser with dispense system rotated vertically.

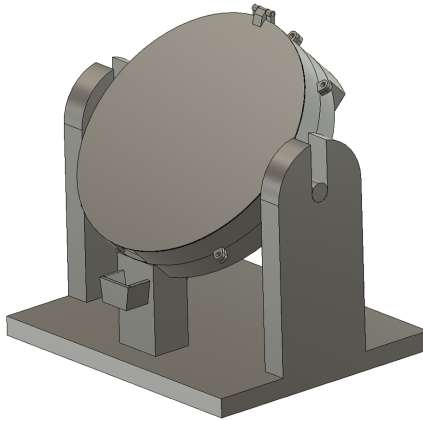


Figure 6: The final design of the Pill Dispenser.

For the final design, the following steps have been taken to address the issue:

1. **Resize the chambers** Chambers size were changed so that they match. The result is device now has a cylindrical form where the chamber mechanisms are located directly on top of each other, maximising the space used
2. **Remove the ramp, change disposal mechanism** The disposal mechanism was also changed. It works in tandem with dispense mechanism, which was also changed. The pills are now dispensed by the same movement by which they are made available, namely by rotation of the dispense mill.
3. **Introduce a separate axis of rotation to dispense pills** We will use tilting movement of the whole device for dispense of the pills into a cup or a hand of a patient, but for that we would have the whole construction elevated.
4. **Expand the device to include the stand** As mentioned above, it needs to be elevated. The stand will provide not only the elevation, but also another axis of rotation, therefore decoupling the disposal and dispense mechanisms from one another.

In the image 6 you can see the final design. It has all the features mentioned above already covered. In the next chapter we will go step by step into the details of the design and see how the proposed solutions were implemented.

3 Construction and 3D-Print of Design

The final design differs significantly from the prototypes. It has seen many iterative improvements and thus deserves its' own chapter to cover all the changes to each component of the design. The final design consists of the following bodies:

1. **Stand** Is the stand upon which the device resides. Needed for introduction an another axis of rotation and decoupling dispense and disposal mechanisms
2. **UpperBody** Consists of a walls and a floor of a dispense mechanism with a cutout for dispense and disposal paths as well as connection mechanisms.
3. **UpperMill** Is a mechanisms that divides the whole dispense system into 21 functional + 1 service chamber (22 in total). Inner side contains a gear and teeth for connecting upper mill with the lower mill (synchronicity between them is still retained)
4. **LowerMill** Is a mechanisms that divides the whole disposal system into 22 chambers. Unlike dispense system, they are all the same, since chambers need to be of the same size as dispense system, but there is no need for the service chamber. it also contains grooves to reduce the surface contact with lower body and thus friction
5. **LowerBody** is a very complex detail. Firstly, it contains cutouts for the stepper motor, driver and the deck that holds further electronics. Secondly, it has "ears" that define the axis of rotation for the dispense pathway. Thirdly, it contains grooves for the lower mill to travel on rotationally, to optimize power spent on overcoming friction.
6. **LowerDeck** Is a cover for all the electronics inside. It contains grooves on the sides for it to slide into electronics chamber.
7. **Electronics** is the housing for microcontroller, battery and charger. it also contains cutout for the deck to slide into so that electronics is covered from the outside.
8. **Holder** is a mechanism of additional mechanical fixation of the electronics. Although the Electronics compartment was designed so, that all the components are fixed in place, it might come to the situations where excess movement (e.g. during transportation) might cause electronics to fall out. this mechanism prevents it.
9. **UpperDeck** Is an upper cover of the device.

Moving forward, we will go step-by-step into the design of each of those components and also will cover the nuances of 3D-printing them all. For now, however, we will compare the final design with the goals set and why this design was chosen in the end. The goals can be divided into 2 types: Initial requirements and the problems arisen during previous designs. Initial requirements are:

1. **The device contains 21 chambers, 3 for each day of the week.** This requirement is satisfied. The final design has 22 chambers, 21 of which can contain pills.
2. **The device contains a pill disposal system.** This requirement is also satisfied.

3. **The device has an accompanying app for remote control.** This is not implemented yet. Alternatively, this requirement tells us, that an user interface on the body of device itself is not required. The electronics of this device are chosen so that this requirement can be implemented later on.

The problems that arisen during the design are solved:

1. From first prototype:

- (a) **Too many moving parts** Final design only has 2 mills that are moving simultaneously on the same axis and nothing else. This is significant improvement over the first design
- (b) **Requires a large dispenser bowl** the disposal system of the final design is located directly underneath the dispensing system and they have the same spatial dimensions.
- (c) **Contains several small objects** The only small object is the holder, all the other ones are big and static, which reduces the amount of potential failure points.

2. From first prototype:

- (a) **Chambers of the upper mechanism are smaller** Since the dispense and disposal mechanisms are of the same size, this is not the problem
- (b) **Synchronous movement of both chambers** While both mechanisms still move synchronically, the pathways of dispense and disposal were decoupled, therefore their synchronous movement is not a problem anymore.
- (c) **Reliance on a ramp for pills to fall down.** The final design doesn't have a ramp. the pills would fall directly down in the disposal pathway. In dispense pathway, the device would be tilted 45 degrees which makes certain that the pills would fall down.

3. From third prototype:

- (a) **Too tall** The device will never be completely vertical. Although the device is now at an elevation it is still shorter as when having to have one wheel always be vertical.
- (b) **Requires more Rotational momentum** since both mills are now horizontal, there is no extra power spent to battle gravity. the mills will only normally rotate in horizontal position.

3.1 Components of the final design

3.1.1 UpperBody

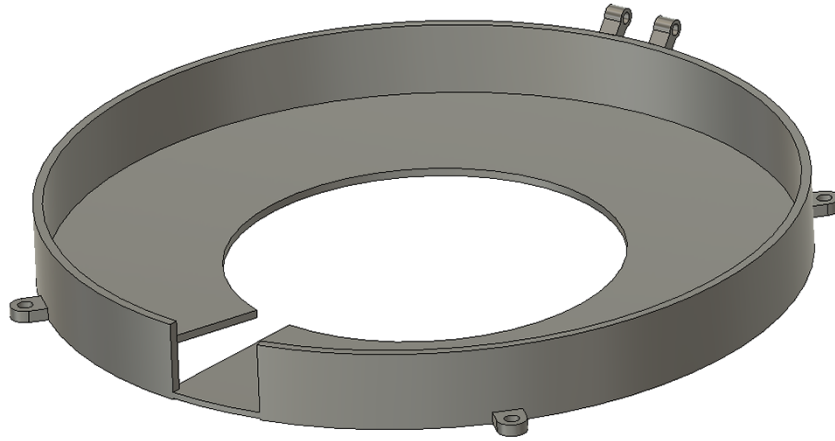


Figure 7: Upper Body

Upper body is a simple design consisting of a cylindrical shape with multiple cutouts. First one is on the floor, through this hole the pills that have not been taken would fall. The other cutout is in the wall, through this cutout the pills that will have to be taken would fall. In the middle there is a circular hole for housing the mill. under the cutout for the dispense pathway, there is a notch to prevent accidental rotation

The measurements are:

- outer radius = 100mm
- inner radius = 52 mm
- height = 20 mm
- thickness of floor = 2 mm
- thickness of wall = 2.5 mm
- notch depth = 0.4 mm

3.1.2 UpperMill

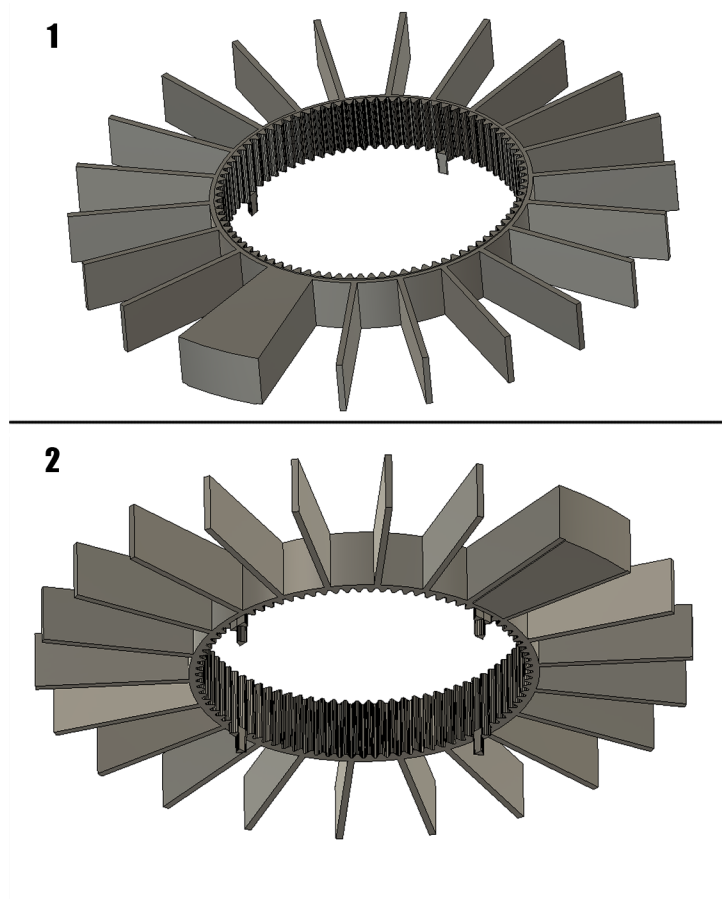


Figure 8: Upper Mill. (1) View from above. (2) view from below

Depicted on image 8, Upper mill consists of 21 functional chambers and one service chamber. the service chamber is needed to cover the hole in the upper body, so that at any time, 21 chambers are available. Inner rim is designed as a cogwheel. It has 88 teeth, it will be important later for the stepper motor movement calculation. The gear was designed using a free tool called DXF and SVG GEAR DXF GENERATOR [4]. These parameters were chosen to match the size of the cog and amount of teeth needed:

- Gear 1 Tooth Count: -88 (internal gear)
- Gear 2 Tooth Count: 32
- Module: 1.11 (mm)
- Pressure Angle: 20°
- Clearance: 0.15 mm
- Gear 1 Center Hole Diameter: 0 mm (no hole)
- Gear 2 Center Hole Diameter: 0 mm (no hole)

The lower side contains 4 teeth to connect the upper mill to the lower mill. This implementation makes sure that the two mills will move at the same time, removing need for a separate stepper motor for each of them or a connection directly to the upper mill.

There are certain design decisions that have been taken to reduce friction. Firstly, on the upper side the ring is offset by 0.4 mm so that the upper deck will not touch the whole mechanism, but only this ring. At the bottom side, the teeth are complex shaped. the upper part of the tooth has a length of 2.4 mm, which is 0.4 mm taller than the thickness of the floor of the upper body. The idea behind it is that the wings of the mill will not touch the floor directly, leaving 0.2 mm opening between the floor and the upper mill.

The measurements of the upper mill are:

- chamber height = 17 mm
- chamber length = 45.05 mm
- inner ring diameter = 100 mm
- outer ring diameter = 193 mm
- gear tooth depth 2.7 mm
- connecting tooth length = 2.4 mm + 5 mm

3.1.3 LowerMill

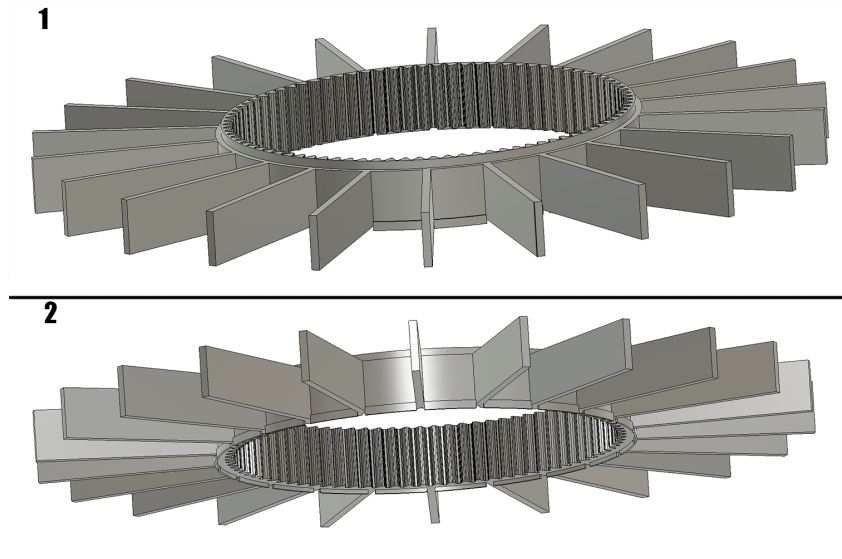


Figure 9: Lower mill. 1 view from above. 2 view from below

Depicted on image 9, Lower mill consists of 22 chambers which are the same. This is done to make sure that the sizes of upper and lower mill chambers are the same. Just as with the upper mill, inner ring is also a cogwheel, designed the same way and with the same parameters as upper mill. Unlike the upper mill, lower mill doesn't have teeth to connect. The teeth of upper mill are inserted into teeth of cogwheel of the lower mill.

The upper part also contains a ring that is 2 mm above the rest of the construction. This ring exists to eliminate the friction between the ceiling (which is a floor of the upper mill) and the chambers.

The lower part is also made in such a way as to reduce friction. it contains rims that would travel rotationally along the ring located on the lower body. they are rather tall, measured at 1 mm to make sure that they don't accidentally fall out during movement.

The measurements of lower mill are:

- chamber height = 13.6 mm
- chamber length = 45.05 mm
- inner ring diameter = 100 mm
- outer ring diameter = 193 mm
- gear tooth depth 2.7 mm

3.1.4 LowerBody

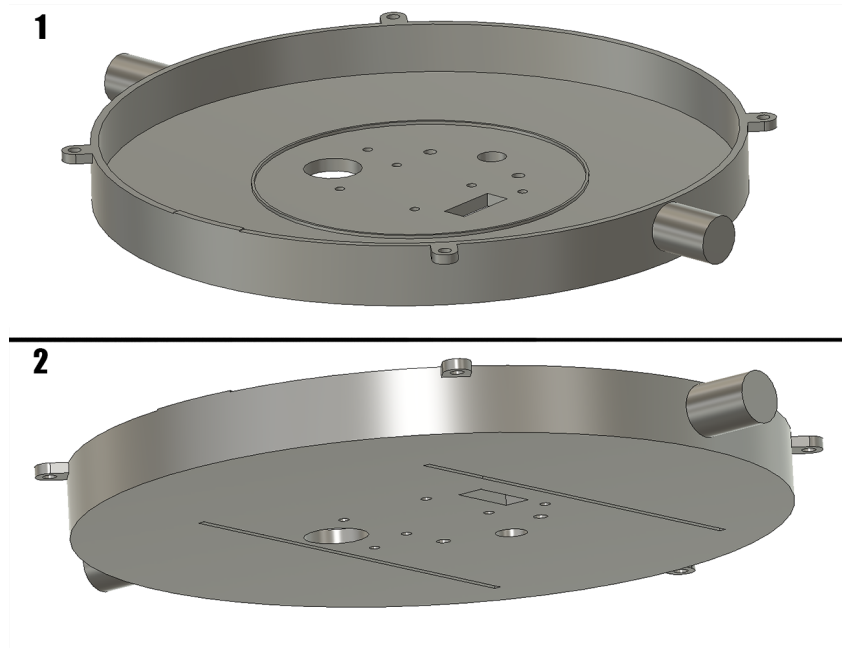


Figure 10: Lower Body. 1 view from above. 2 view from below

Depicted on image 10, Lower body is a bit more complex than the upper one. It is designed to connect multiple electronic components, Stepper motor, motor driver - directly, charger and microcontroller - indirectly, through the lower deck.

On the inner side there is a ring around which the lower mill rotates. This ring also separates the chamber area and electronics area. Inside this ring there are cutouts for stepper motor, driver and wires.

On the outer side, there are "ears" upon which the whole device would be positioned on a stand. These ears define the axis of rotation for the dispense pathway. Outer rim contains a ridge to fix the upper body and prevent accidental rotation.

On the inner side there are also grooves to append lower deck, these grooves are needed to remove the possibility of attaching the lower deck the wrong way. Grooves are 125 mm long, 2 mm wide and 2 mm deep.

The measurements of lower body are:

- radius = 100mm
- ring inner radius = 50.5 mm
- ring inner radius = 51.4 mm
- ring height = 1 mm
- body height = 20 mm
- thickness of floor = 4 mm
- thickness of wall = 2.5 mm
- notch depth = 0.4 mm

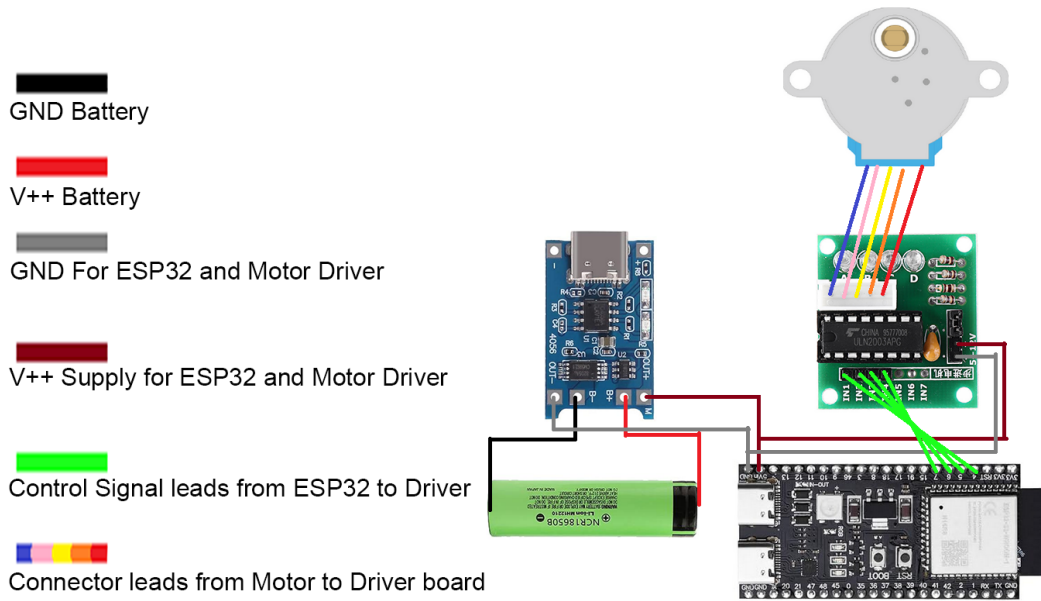


Figure 11: Connection schema of electronic components. Components from top to bottom: 1) 28BYJ Stepper motor 2) Left: Charger Module 3) Right: ULN2003 Motor Driver 4) Left: Panasonic NCR18650B Battery 5) Right ESP32 S3 DevKitC-1 N16R8 S3 Micro-controller

3.1.5 Electronics

The device needs a microcontroller and a stepper motor for control. We need to know what we need for our electronics to filter out possible choices. Microcontroller needs to be able to control a stepper motor and also to have a network interface, Wi-Fi or Bluetooth, so that we can host an user interface somewhere else. Stepper motor needs to have enough torque to rotate 2 mills and a load. We don't need a high performance stepper motor, since our rotational speed is not that important. also it should be noticed that there is a gear reduction of 4:1 on the gears which results in 4 times higher torque for 4 times lower speed. From the measurements above we know that the diameter of mill is 200 mm, the weight is hard to calculate, but we can access the calculation used by the slicer software. The results are 44 g for Lower mill, and 61 g for Upper mill for 105g in total. The extra weight from pills would be around 100g for 205g in total. While there has been no calculation involved, the setup was tested with the coins to serve as a load, however it was tested only with the lower mill attached. We will come back to discuss the results in the later part of the document. Considering also that the one of the goals of the project was to develop the device for as cheap as possible, a 28BYJ [5] stepper motor was chosen, together with the ULN2003[6] Motor driver.

For the microcontroller, an ESP32 S3 DevKitC-1 N16R8 was selected [7]. it is a common choice to pair this microcontroller and stepper motor and it has the network interface (both Wi-Fi and Bluetooth). For us a BLE is of particular interest. It allows us to save battery, since we don't need a constant stream of data exchange between a smartphone and device.

Another addition to the electronics is inclusion of battery. While not a requirement, battery-powered device is much more convenient to use as it makes it independent from being located near a socket. For this, Panasonic NCR18650B [8] Battery was selected. Image 11 Shows the connection schema of selected components.

3.1.6 Lower Deck(Electronics compartment)

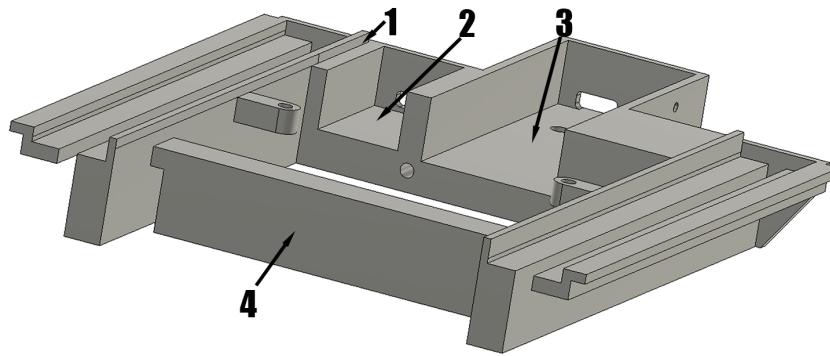


Figure 12: Lower Deck and its' features: (1)Ridge (2)Microcontroller housing (3)Charger Module housing (4)Battery Wall

The lower deck houses three electronic components: Microcontroller, battery charger and battery. It also serves as a cover for the stepper motor, whereas the driver would be located on the opposite side of the lower body, inside the ring. The shape of the lower deck is quite more complex than that of the previous bodies, therefore it makes sense to divide the description of it into multiple parts. on the image 12 you can see the lower deck and a numbers that point to the parts of components. Here is a description of all the components of this body:

1. **Ridge** Lower body has 2 parallel grooves on its lower part. These grooves will house these ridges on the lower deck. The ridges are designed to actually be a tiny bit smaller (1.6 mm vs 2 mm) than the notches of the lower body, we will discuss why in the later chapter. Ridges are 125mm long, 1.6 mm thick and 2 mm deep.
2. **ISP32 Microcontroller housing** This is where the microcontroller would be placed. We know the dimensions of the microcontroller (63.5 mm x 28 mm) and this opening is made to be around the same size to fit it (67.6 mm x 28 mm). It has cutouts on the body for 2 USB ports present on a microcontroller, as well as a LED.
3. **Charger Module housing** In this part the charger circuit will be nested. The cutout matches exactly the dimensions of the charging module (28 mm x 17.5 mm). This compartment contains also the cutout on the body for the USB port.
4. **Battery Wall** Battery holder is attached to this wall. at the sides one can notice that the connection to the side walls does not go all the way. The reason is that wires will go from the battery holder through these openings.

Lower deck also features a long hole for a slide-in lid which you can see to the sides from the Ridges that connect Deck to the lower body.

3.1.7 Stand

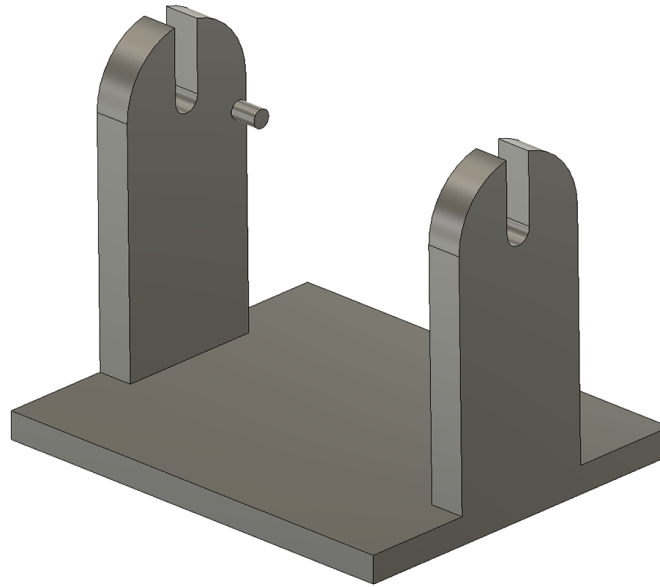


Figure 13: Stand

Depicted on 13, Stand is a simple design that features 2 distinct components: Pillars to the sides to hold the device and a dispensing dock in the front and center of it. The pillars have a cutout to slide down the device into it and notches to prevent the device from tilting backwards. The dispensing dock is designed to match the shape of the device so that it can easily be tilted into it at a 45 degree angle. The dimensions are:

- Stand height from bottom to the peak of stand: 190 mm
- Stand Width: 240 mm
- Stand Length: 200 mm
- Stand floor thickness: 15 mm
- Pillar thickness: 20 mm

It should be noted, however, that although this Stand is the default solution for the sake of completeness of the project, ideally, it would be manufactured not with a plastic, but rather with metal. The purpose of the stand (besides providing pathway for dispense of the pills) is also to provide stability of the construction so that people suffering from tremors would be able to lean firmly onto it and stabilize their hand. In this case, plastic might not be the optimal solution.

3.2 3D printing specifics

As mentioned earlier, one of the goals of the project is to make the device buildable with consumer-grade devices, among them 3D-Printers. This requirement introduces first physical limitations for the device, as 3D-Printers have a limited printing space. For this project as a reference printer, a Prusa MK4S 3D-Printer [9] was selected. From the Specification we can see that the build volume is 250 x 210 x 220 mm, putting a limitation on how large our device can be. Theoretically, of course, we could circumvent this limitation by designing the components in such a way that they would consist of multiple parts interlocking with themselves, however this adds unnecessary complexity to an already extensive project. The dimensions of each component mentioned in previous section conform to this limitation.

As for material selection, Prusa MK4S (and many other printers) support printing with Polylactic Acid (PLA) plastic. This material has many advantages: it is biodegradable (it is a polymer of lactic acid), lightweight, sterile [10], doesn't produce as much fumes (but still does) as standard Acrylonitrile-Butadiene-Styrene (ABS) plastic [11], due to lower printing temperature and is easy to print with. However, certain disadvantages should be mentioned: Heat deflection at around 55-60 degrees [12] means it cannot be washed with boiling water as it would warp the body. Pure PLA is food safe [13], however the pigments might not be, so any wear might introduce microparticles that might not be as safe to consume if colored PLA is chosen.

Printing components that are exactly equally sized (e.g. a mill with diameter of 180mm to fit inside a ring with inner diameter) has a certain problem associated with it, which must be considered when designing and printing the device. The current models have already had this issue addressed: all interlocking components are made with certain tolerance (1mm for Mills, 0.2mm for Lower Deck). The Printer has nozzle diameter of 0.4mm [9], which means that horizontal (x-y axis) resolution is exactly that. It cannot print features smaller than its nozzle diameter, therefore, usage of small body features, extrusions etc was avoided during design process.

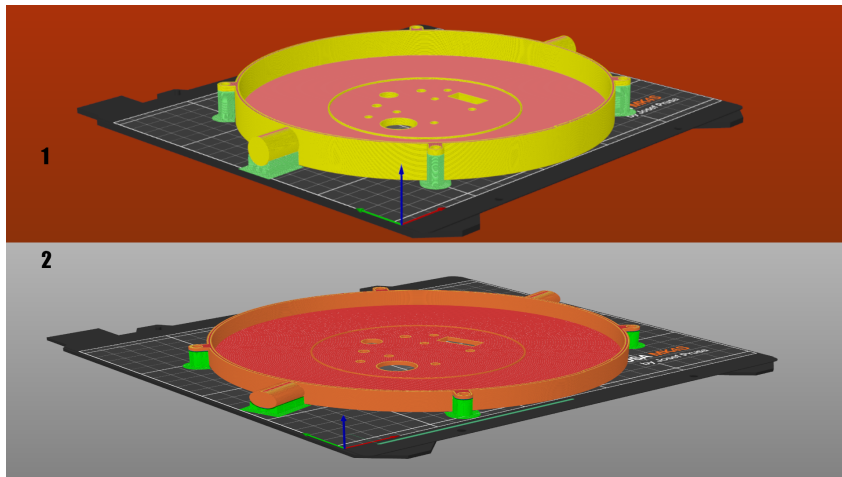


Figure 14: Issue when slicing. On the upper image (1) the supports are out of printing bounds, as the ominous red aura highlights at this problem. On the lower image (2) the problem is fixed. The component was rotated, so that the sticking-out parts of it are given a certain distance from the border to be able to draw support material

4 Microcontroller Programming

Note: For programming part, all the source code is available under authors Github Repository here [14]. Code for this section to be found in the **ESP32/Pillspender** folder of the repository.

As mentioned before, the microcontroller of choice is an ESP32 S3 DevKitC-1 N16R8 [7]. This microcontroller has 16MB Octal Serial Peripheral Interface (SPI) flash memory and 8MB of Pseudostatic RAM (PSRAM). It supports Arduino framework, which makes it easier to program, compared to many libraries published by the developers (such as Espressif's own ESP-IDF) as it abstracts away some low-level programming through helper libraries (such as **stepper.h** for charging/discharging of motor coils). The code consists of 2 separate parts: The first part is responsible for communication with stepper motor. It means that it covers all the functions required for stepper motor to rotate a necessary amount of steps to move the next chamber into position. The second part is responsible for BLE interface and establishing the device as a BLE host.

The **main.cpp** file is rather minimal because the code was semantically split also in **BLE** and **Motor Control** parts. Here is a short overview of the contents of main.cpp file:

```
1      #include <Arduino.h>
2      #include "motorcontrol.h"
3      #include "BTooth.h"
4
5      void setup() {
6          Serial.begin(115200);
7
8          motorSetup();          // stepper + pins
9          bleSetup();            // BLE GATT server
10     }
11
12     void loop() {
13         bleLoop();
14         MotorStatus motorStatus = motorLoop(); // Call motorLoop and
           ↳ get its status
15         if (motorStatus == MOTOR_JUST_COMPLETED) {
16             bleNotifyDispenseComplete();
17         }
18         delay(10);
19     }
```

Listing 1: Main Arduino Loop Code

As seen in the code block 1, **main.cpp** file includes 3 libraries, 2 of which are custom (**motorcontrol.h** and **BTooth.h**) and one is a built-in **Arduino.h** library. Specifics of Arduino microcontroller programming is that main body is divided into **setup()** and **loop()** functions, unlike standard C/C++ file where only **int main()** exists. Effectively, this is the same thing as C code having a **while(TRUE)** loop inside the **main()** function.

4.1 Stepper motor controls

Controls of a stepper motor are handled by 2 files: **motorcontrol.cpp**, which contains the implementation itself, and **motorcontrol.h**, which contains public interface of the motor control module. The motor control implementation is split into 2 (.cpp and .h) files

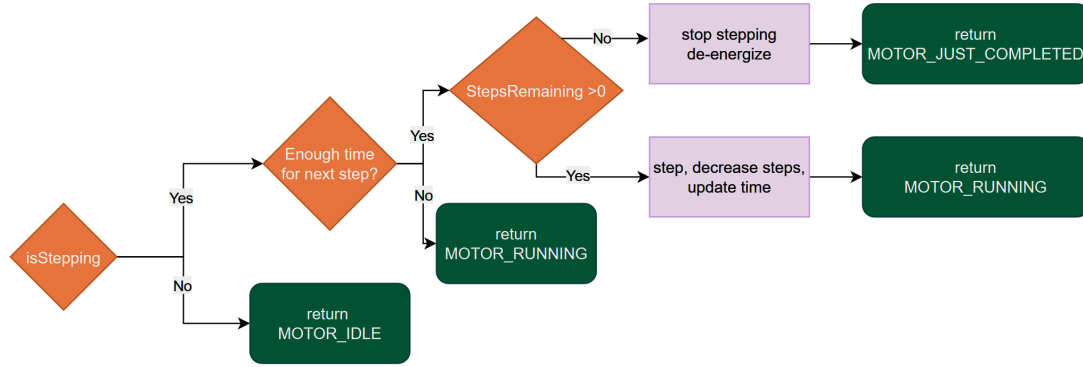


Figure 15: The flow chart of stepper motor control logic within the `motorLoop()` function. This function is called repeatedly within `loop()` function of the `main.cpp` file.

is not only a stylistic choice (splitting declaration action from definition action) but also positively affects code readability, compilation time and prevents accidental redefinition of functions. For example this code block 2 shows the contents of the `motorcontrol.h` file. The first 2 lines (`#ifndef...#define...`) exist as a preprocessor check (or include guard, a term often used in the industry), they ensure that the declarations within this header file are processed only once per compilation. If `MOTORCONTROL_H` has not been defined during the compilation of a particular `.cpp` file, the preprocessor includes the subsequent lines. These lines contain the declarations for other parts of the program to interact with the motor control module, such as function prototypes (e.g., `void motorSetup()`), enumeration definitions (`enum MotorStatus`), and external variable declarations (`extern const int OneChamber`). The actual implementations (definitions) for these prototyped functions are in the corresponding `motorcontrol.cpp` file, which is compiled separately and later linked with machine code generated from other files.

```

1  #ifndef MOTORCONTROL_H
2  #define MOTORCONTROL_H
3
4  #include <Arduino.h>
5
6  extern const int OneChamber;           // 1/8 revolution,
    ↪ exported for BLE layer
7  enum MotorStatus { MOTOR_IDLE, MOTOR_RUNNING,
    ↪ MOTOR_JUST_COMPLETED };
8  void motorSetup();                    // call once from setup()
9  bool motorStartDispense(int numSteps); // returns true if motor
    ↪ accepted the job
10
11  MotorStatus motorLoop();
12  // ...
13
14  #endif // MOTORCONTROL_H

```

Listing 2: Header for Motor Control Interface

Having covered the structure of the files responsible for handling motor controls, it is time to look at an actual implementation within the `motorcontrol.cpp` file, as seen in this code block 3. The file begins with inclusion of libraries. `motorcontrol.h` is the accompanying header file containing function prototypes, `Stepper.h` library is a built-in Arduino library for high-level stepper motor control.

After libraries come 2 *const* variables needed to calculate rotation (code block 3, lines 4 and 5). According to the 28BYJ-48 documentation [5], it has Speed Variation Ratio 1/64 and Stride Angle 5.625deg /64 (it is usually denoted for half-step mode, so full-step mode would be 1/32), which essentially means that inside this motor, there is an internal gear connected to an external gear that rotates the shaft, connected with a 1-to-64 gear reduction, but the internal gear also steps 1/64th of a rotation each time coil polarisation changes. It means that total amount of steps for the full rotation of the shaft is $64 \times 64 = 4096$, which gives us the value for the `const int stepsPerRevolution = 2048;`. To calculate `const int OneChamber = stepsPerRevolution / 8;`, gear ratio of designed device should be taken into consideration. As mentioned in the previous section, the inner ring of the Mill contains 88 teeth, while gear that is attached to the stepper motor has 32 teeth. Since there are 22 Chambers in total, each chamber corresponds to exactly 4 teeth of rotation (teeth are equal for both motor and Mill gears).

Next is the definition of the pins that are attached to the wires that drive the stepper motor (lines 7 to 10) and then (line 12) **Stepper** class of the **Stepper.h** library. It takes as an input amount of steps and energizing leads. After the class, helper variables are declared. They serve utility functions such as state change in the code. We will review their purpose when they appear in functions.

After establishing the structure, the functions are declared:

- **deenergizeStepper()** function that depowers wires when they are not in use, since methods of the **Stepper** class don't do that. It is an important function for saving energy, as constant current to the coils will drain battery charge very fast.
- **motorSetup()** function initializes the motor. Since it only takes speed as an input, there is not much else to initialize, but it also has output lines to check whether operation has been correctly done.
- **motorStartDispense(int numSteps)** function serves as the primary interface to initiate a motor rotation. It rotates the Pill Dispenser exactly one chamber. It has a precondition `if (!isStepping)`: it will not do anything if motor is already in process of rotating. It takes an integer `numSteps` as input, specifying how many steps the motor should turn.

Next, **MotorStatus motorLoop()**. The **MotorStatus** is a function of an *enum* type, which means it can only a predetermined set of values. It which serves the purpose of state-selector. It is defined in the **motorcontrol.h** file (seen here: 2, on line 7). The logic for state-change is defined in the **motorLoop()** variable. Graphically, the controlling logic can be seen on image 15. The loop works as follows:

- **Top-level availability test**

The `if (isStepping)` decides whether the motor currently has work to do.

- `false` $\Rightarrow$ return `MOTOR_IDLE`.
- `true` $\Rightarrow$ proceed to timing check.

- **Timing check**

Evaluate `millis() - lastStepTime  $\geq$  stepInterval`.

- `false` (interval not yet elapsed) $\Rightarrow$ return `MOTOR_RUNNING` to signal the dispense is ongoing but no step is taken in this call.

- true (interval elapsed) $\Rightarrow$ readiness for a potential step.
- **Step-or-finish decision**
Check `stepsRemaining > 0`.
 - `stepsRemaining > 0`:
 - * execute `myStepper.step(1)`;
 - * decrement `stepsRemaining`;
 - * update `lastStepTime`;
 - * return `MOTOR_RUNNING`.
 - `stepsRemaining  $\leq$  0`:
 - * set `isStepping` to false;
 - * call `deenergizeStepper()`;
 - * return `MOTOR_JUST_COMPLETED`.

Note that it is not a recursive loop, it doesn't call itself. This function is called from `void loop()` function of the `main.cpp` file.

```

1  #include "motorcontrol.h"
2  #include <Stepper.h>
3
4  const int stepsPerRevolution = 4096;
5  const int OneChamber        = stepsPerRevolution / 8;
6
7  const int in1Pin = 4;
8  const int in2Pin = 5;
9  const int in3Pin = 6;
10 const int in4Pin = 7;
11
12 static Stepper myStepper(stepsPerRevolution, in1Pin, in2Pin, in3Pin,
    ↪ in4Pin);
13
14 static bool          isStepping      = false;
15 static int           stepsRemaining  = 0;
16 static unsigned long lastStepTime    = 0;
17 static const unsigned long stepInterval = 2; // ms
18
19 static void deenergizeStepper() {
20     digitalWrite(in1Pin, LOW);
21     digitalWrite(in2Pin, LOW);
22     digitalWrite(in3Pin, LOW);
23     digitalWrite(in4Pin, LOW);
24     Serial.println("Stepper coils de-energized.");
25 }
26
27 void motorSetup() {
28     Serial.println("Initializing Stepper Motor...");
29     myStepper.setSpeed(10);
30     Serial.println("Stepper Motor Initialized.");
31 }
32
33 bool motorStartDispense(int numSteps) {
34     if (!isStepping) {
35         Serial.printf("Starting dispense of %d steps.\n", numSteps);

```

```

36         stepsRemaining = numSteps;
37         isStepping     = true;
38         return true;
39     }
40     return false;
41 }
42
43 MotorStatus motorLoop() {
44     if (isStepping) {
45         if (millis() - lastStepTime >= stepInterval) {
46             if (stepsRemaining > 0) {
47                 myStepper.step(1);
48                 stepsRemaining--;
49                 lastStepTime = millis();
50                 return MOTOR_RUNNING;
51             } else {
52                 isStepping = false;
53                 deenergizeStepper();
54                 Serial.println("Dispense complete.");
55                 return MOTOR_JUST_COMPLETED; // Signal completion
56             }
57         }
58         return MOTOR_RUNNING; // Still stepping or waiting for
           ↪ interval
59     }
60     return MOTOR_IDLE; // Not stepping
61 }

```

Listing 3: Implementation of Motor Control Interface

4.2 Bluetooth Controls

Similarly to stepper motor controls, Bluetooth controls have also been split into 2 files, **BTooth.h** and **BTooth.cpp**. the header file **BTooth.h** is quite small, as you can see in the code block 4, it only contains 8 lines, main body of which contains 3 prototype functions, which will be described when the implementation file, **BTooth.cpp** is covered.

```

1  #ifndef BT00TH_H
2  #define BT00TH_H
3
4  void bleSetup();
5  void bleLoop();
6  void bleNotifyDispenseComplete();
7
8  #endif // BT00TH_H

```

Listing 4: Header for Bluetooth implementation

Before doing so, the fundamentals of Bluetooth, and particularly BLE must be covered. Bluetooth standard was initially covered under IEEE 802.15.1 Task group [16], which defined the physical layer and Media Access Control (MAC) (layers 1 and 2 of the Open Systems Interconnection (OSI) model, respectively) implementation of this communication protocol. Currently it is maintained by Bluetooth Special Interest Group (SIG), which has developed a BLE extension protocol, which is used in this thesis. The functional principles of this protocol are thoroughly documented in the Bluetooth Low Energy Primer[15]. All definitions will be taken from this document, with additions on

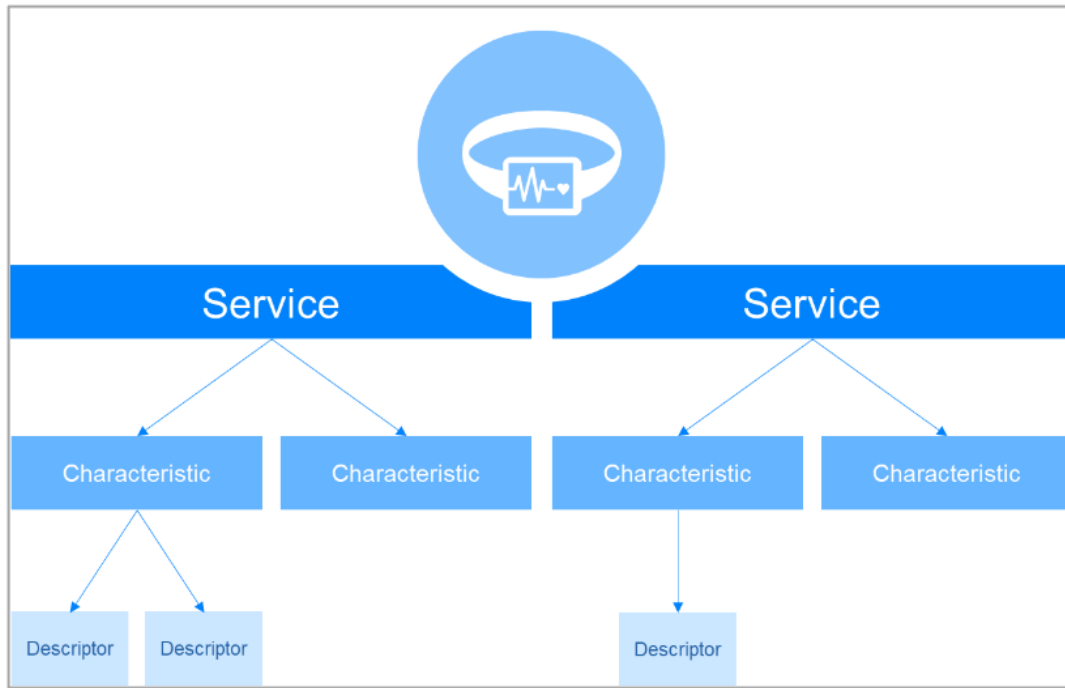


Figure 16: Generic Attribute Profile hierarchy shows the structure of service, characteristics and descriptors. Credit [15]

how they relate to this project. To cover the implementation of BLE communication of device with an Android App, some definitions need to be established:

- **Bluetooth Low Energy** is a wireless communication protocol designed for short-range communication with very low power consumption, making it ideal for battery-operated devices.
- **Central (Client)** is the device that initiates a connection and requests data. In this case, this is the Android App. Centrals scan for peripherals.
- **Peripheral (Server)** is the device that advertises its presence and provides data. In this case, this is ESP32 Pill Dispenser. Peripherals wait for a central to connect.
- **Advertising** is the process by which a peripheral broadcasts small packets of data to announce its presence, services, and device name. This allows centrals to discover it.
- **Scanning** is the process by which a central listens for advertising packets from peripherals.
- **Connection** is the establishment of a link between a central and a peripheral after discovery through advertising and scanning.
- **Device Name** is a human-readable name a device advertises, in this case it is "PillDispenserESP32".
- **Device Address (MAC Address)** is a unique hardware identifier for a Bluetooth device.

The main unit of communication and crucial part of the protocols structure is the Generic Attribute Profile (GATT) profile. While GATT is part of BLE protocol, it has many terms associated directly with it, as seen on image 16. The associated terms are described as follows:

- **Generic Attribute Profile** or GATT, Defines high level data types known as services, characteristics and descriptors in terms of underlying attributes in the attribute table.
- **Profile** Profile specifications define the roles that related devices (like the smart-watch and key fob) assume and in particular, define the behaviour of the client device and the data on the connected server that it should work with. In case of this thesis, a custom profile has been implemented, through the services and characteristics that have been defined.
- **Service** A service specification defines a single service along with the characteristics and descriptors that it contains. The behaviours to be exhibited by the device hosting the service in response to various conditions and state data values are defined in the service specification.
- **Characteristic** Characteristics are individual items of state data and have a type, an associated value and a set of properties which indicate how the data may be used in terms of sets of related GATT procedures.
- **Value** The attribute value field is a byte array contain the attribute's value. Interpretation of the byte array both in terms of data types and semantics is the concern of higher layers of the stack.
- **Properties** Define what operations a client can perform on a characteristic's value (Read, Write, Write Without Response, Notify, Indicate).
- **UUID** all services, characteristics and descriptors have a Universally Unique Identifier (UUID) which identifies the type of the service, characteristic or descriptor to which it relates rather than a particular instance on a particular device.
- **Descriptor** Descriptors belong to some characteristics and can contain metadata like a textual description for the characteristic or might provide some means of controlling the behaviour of a characteristic. Characteristics have zero or more descriptors attached to them.
- **CCCD** Any characteristic supporting notifications or indications must have a client characteristic configuration descriptor because its value (which clients may write to) controls whether or not notifications or indications are currently enabled or not.

Additionally, all BLE Operations can be categorized in 3 groups:

- **Read Operation** when the central requests the value of a characteristic from the peripheral
- **Write Operation** when the central sends data to the peripheral to set the value of a characteristic
- **Notification Subscription** when the central enables notifications on a characteristic (by writing to its Client Characteristic Configuration Descriptor (CCCD)) to receive updates when its value changes.

4.2.1 Implementation of Bluetooth Protocol

Next, the actual implementation of BLE interface shall be covered. The code block 5 provides the definition and initialization part. Firstly, preprocessor directives to include other files:

- `BTooth.h` is the header file for this file;
- `motorcontrol.h` is used for manual dispense;
- `Arduino.h` is used extensively for many tasks such as `Serial.println()`
- `NimBLEDevice.h` is the header for the NimBLE Arduino library, which provides Bluetooth Low Energy (BLE) functionality for ESP32;
- `Preferences.h` provides an easy way to use the ESP32's non-volatile storage to save data that persists even after the ESP32 is restarted or powered off

After that, Service and Characteristic UUIDs are declared. Their exact values are not as relevant as long as they are unique and existing. What is relevant, however, is what purpose they serve. This is important because BLE protocol has limited capacity for these UUIDs, which can lead to issues, such as Android App failing to fetch all characteristics. The implemented characteristics serve this purpose:

Writable

- `Set_Device_Time` — synchronize the ESP32 clock.
- `Set_Dispense_Schedule` — add, edit, or delete schedule entries.
- `Trigger_Manual_Dispense` — manually trigger a dispense (logged for traceability).

Readable

- `Get_Device_Time` — read the current device time.
- `Get_Dispense_Schedule` — retrieve the stored schedule.
- `Get_Last_Dispense_Info` — timestamp + status of the last event.
- `Get_Time_Until_Next_Dispense` — countdown to the next dose.
- `Get_Dispense_Log` — full dispense history.

As seen in the code block 5, there is a naming convention in place. For UUIDs it is `CHAR"_CHARACTERISTIC_PURPOSE"_UUID` except for `SERVICE_UUID` which is a service and not a characteristic, therefore it doesn't have `CHAR` prefix. The pointers to these characteristics are initialized as null-pointers i.e. without an address, as not doing so will make them point to random areas of memory, causing unpredictable behavior. `Static` means that they can be accessed anywhere within this file, but cannot be accessed from without. The naming convention for pointers is `p"CharacteristicPurpose"Characteristic`. Next, `static Preferences preferences;` class is declared to manipulate static memory and finally a `static bool deviceConnected = false` boolean (can have values either 'true' or 'false') variable is initialized to track the connection status.

```

1  #include "BTooth.h"
2  #include "motorcontrol.h"
3
4  #include <Arduino.h>
5  #include <NimBLEDevice.h>
6  #include <Preferences.h>
7
8  // BLE Service + Characteristic UUIDs
9  #define SERVICE_UUID
10     ↪ "03339647-3f4e-43df-abff-fac54287cf1a"
11 #define CHAR_SET_DEVICE_TIME_UUID
12     ↪ "65232f1d-618a-4268-9050-0548142a4536"
13 #define CHAR_SET_DISPENSE_SCHEDULE_UUID
14     ↪ "999c584e-06c0-49a1-995a-66b7c802ac1b"
15 #define CHAR_TRIGGER_MANUAL_DISPENSE_UUID
16     ↪ "36bb95f2-e57e-4db9-b9aa-fb6541ee784e"
17 #define CHAR_GET_DEVICE_TIME_UUID
18     ↪ "272ee276-e37e-4d78-8c5e-bb7225d35074"
19 #define CHAR_GET_DISPENSE_SCHEDULE_UUID
20     ↪ "b53c2ed4-ae26-476d-8414-011a025dddfc"
21 #define CHAR_GET_LAST_DISPENSE_INFO_UUID
22     ↪ "40d3b5d8-5480-4b7b-a115-5fe86bf17d7d"
23 #define CHAR_GET_TIME_UNTIL_NEXT_DISPENSE_UUID
24     ↪ "4b14acc4-768a-43e1-9d6c-0d97307e2666"
25 #define CHAR_GET_DISPENSE_LOG_UUID
26     ↪ "6f182da7-c5a8-40ab-a637-f97ed6b5777b"
27
28 static NimBLECharacteristic *pSetDeviceTimeCharacteristic =
29     ↪ nullptr;
30 static NimBLECharacteristic *pSetDispenseScheduleCharacteristic =
31     ↪ nullptr;
32 static NimBLECharacteristic *pTriggerManualDispenseCharacteristic =
33     ↪ nullptr;
34 static NimBLECharacteristic *pGetDeviceTimeCharacteristic =
35     ↪ nullptr;
36 static NimBLECharacteristic *pGetDispenseScheduleCharacteristic =
37     ↪ nullptr;
38 static NimBLECharacteristic *pGetLastDispenseInfoCharacteristic =
39     ↪ nullptr;
40 static NimBLECharacteristic *pGetTimeUntilNextDispenseCharacteristic
41     ↪ = nullptr;
42 static NimBLECharacteristic *pGetDispenseLogCharacteristic =
43     ↪ nullptr;
44
45 static Preferences preferences;
46
47 static bool deviceConnected = false;

```

Listing 5: Preprocessor directives and definitions in the BTooth.cpp file

The next part, the **MyCharacteristicCallbacks** class is a crucial part of the functionality of the code. It determines what actions should be taken when there is a write command coming from the BLE client. This code block 6 contains the implementation of **MyCharacteristicCallbacks** class. This class works like this: it is inherited from the base **NimBLECharacteristicCallbacks** class, which contains functions for various characteristic events (such as read, write, notify subscription changes). Inside this class, there is a single `onWrite(NimBLECharacteristic *pCharacteristic)`

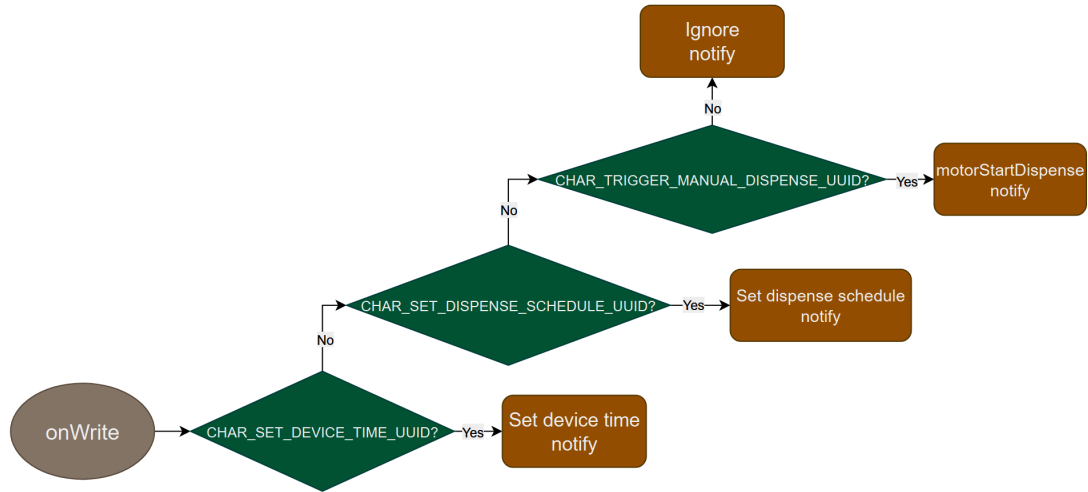


Figure 17: Flowchart of the onWrite method of MyCharacteristicCallbacks class shows the logic under which this method operates. Notify means that whoever was subscribed to updates to a certain Characteristic will get a signal that the value has changed.

`override` method, which overrides (essentially redefines what this method does) the built-in method from base class. It takes as an input `pCharacteristic`, a pointer to the `NimBLECharacteristic` type object that received write request, which determines which characteristic and what value was written. This method takes this input object, which contains UUID and Value, and converts them to string (lines 3 and 4). Then it goes through a list of writable characteristics and compares their UUIDs to the value it got from the input to what exactly needs to be done. A good analogy to this would be a mailman distributing letters. The letters have content which only the correct receiver correctly interprets and acts upon. In this analogy, a mailman is the method `onWrite`, receivers are UUIDs and letters themselves are values. When the correct recipient is found the tasks are done according to who received it:

- **CHAR_SET_DEVICE_TIME_UUID** Sets device time to that of the value of received data and notifies subscribers that value has changed.
- **CHAR_SET_DISPENSE_SCHEDULE_UUID** Sets the schedule to that of the value and save it to Non-Volatile Storage (NVS). Notifies subscribers that value has changed.
- **CHAR_TRIGGER_MANUAL_DISPENSE_UUID** checks the current state of the motor, if it is available for the job, commands it to rotate 1 chamber.

```

1  class MyCharacteristicCallbacks : public
    ↳ NimBLECharacteristicCallbacks {
2      void onWrite(NimBLECharacteristic *pCharacteristic) override {
3          std::string uuid_str =
    ↳ pCharacteristic->getUUID().toString();
4          std::string value_str = pCharacteristic->getValue();
5
6          if (uuid_str == CHAR_SET_DEVICE_TIME_UUID) {
7              preferences.putString("devTime", value_str.c_str());
8              if (pGetDeviceTimeCharacteristic) {
9                  pGetDeviceTimeCharacteristic->setValue("Time Set
    ↳ ACK: " + value_str);
  
```

```

10         pGetDeviceTimeCharacteristic->notify();
11     }
12     } else if (uuid_str == CHAR_SET_DISPENSE_SCHEDULE_UUID) {
13         preferences.putString("schedule", value_str.c_str());
14         if (pGetDispenseScheduleCharacteristic) {
15             pGetDispenseScheduleCharacteristic->setValue(value_str);
16             pGetDispenseScheduleCharacteristic->notify();
17         }
18     } else if (uuid_str == CHAR_TRIGGER_MANUAL_DISPENSE_UUID) {
19         // Attempt to start the motor; returns false if busy
20         if (motorStartDispense(OneChamber)) {
21             if (pGetLastDispenseInfoCharacteristic) {
22                 unsigned long now = millis();
23                 String msg = "Dispensing... Started at: " +
24                     ↪ String(now);
25                 pGetLastDispenseInfoCharacteristic->setValue(msg.c_str());
26                 pGetLastDispenseInfoCharacteristic->notify();
27                 preferences.putString("lastDisp", msg.c_str());
28             }
29         } else {
30             if (pGetLastDispenseInfoCharacteristic) {
31                 pGetLastDispenseInfoCharacteristic->setValue("Dispense
32                     ↪ command ignored: busy");
33                 pGetLastDispenseInfoCharacteristic->notify();
34             }
35         }
36     }
37 }
38 };

```

Listing 6: MyCharacteristicCallbacks class responsible for handling write commands from the server

The **MyServerCallbacks** class is very simple in implementation and handles connection and disconnection of the device. As seen in code block 7, this class inherits from **NimBLEServerCallbacks** and overrides **onConnect** and **onDisconnect** methods. When device connects, it sets **deviceConnected** flag to 'true', meaning that it has connected. When device disconnects, it sets this flag to 'false' and starts advertising.

```

1  class MyServerCallbacks : public NimBLEServerCallbacks {
2      void onConnect(NimBLEServer*) override { deviceConnected = true;
3          ↪ }
4      void onDisconnect(NimBLEServer*) override {
5          deviceConnected = false;
6          NimBLEDevice::startAdvertising();
7      }
8  };

```

Listing 7: MyServerCallbacks class responsible for device connect/disconnect

The function **bleNotifyDispenseComplete()** is designed to perform two main tasks when a dispensing operation has just finished: to inform the connected client that dispense is complete. Then, to store this completion information on the ESP32 NVS. As seen in the code block 8,

```

1  void bleNotifyDispenseComplete() {
2      if (pGetLastDispenseInfoCharacteristic) {
3          unsigned long completeTime = millis();

```

```

4      String completionMessage = "Dispense complete. Finished at:
      ↳ " + String(completeTime);
5      pGetLastDispenseInfoCharacteristic->setValue(completionMessage.c_str());
6      pGetLastDispenseInfoCharacteristic->notify();
7      preferences.putString("lastDisp", completionMessage.c_str());
8      Serial.println("BTooth: Last dispense info (complete) saved
      ↳ to NVS.");
9  }
10 }

```

Listing 8: bleNotifyDispenseComplete function responsible for information logging

Next, comes the **bleSetup()** function. This is a critical function that is responsible for initializing and configuring everything related to BLE communication for the device. As seen in the code block 10, first, it initializes the **Preferences** library (line 2) which is responsible for writing to the NVS, then it initializes the NimBLE stack (line 4), setting the name of the device for advertising. Next, it creates a GATT server instance (line 5) and assigns a callback handler to the server, which handles connection and disconnection of device. Then, it creates BLE service within this server (line 8) with the service UUID already predefined. After that, it starts creating characteristics. Characteristic creation follows a pattern: each characteristic is created in 3 steps:

- **pSetDeviceTimeCharacteristic = pService->createCharacteristic** associates a characteristic with previously created service, establishing a hierarchy (characteristic is subordinate to service). As an input parameters, **createCharacteristic** function takes UUID and BLE properties (such as read, write, notify etc.)
- **pSetDeviceTimeCharacteristic->setCallbacks(new MyCharacteristicCallbacks());** is for characteristics with 'WRITE' property assigned to them. It assigns the **MyCharacteristicCallbacks** handler. The **onWrite** method in this class will be called when a client writes a value to this characteristic.
- **pGetDispenseScheduleCharacteristic->setValue()** is for characteristics with 'READ' or 'NOTIFY' properties assigned to them. It assigns the default value to them.

loadedTime, **loadedSchedule** and **lastDispense** strings are used to retrieve saved values from NVS. After defining all characteristics within the service, **bleSetup** function starts the service to make it and its characteristics active and accessible over BLE (line 24). Finally it set-ups advertising: includes main service UUID to the advertisement, enables scan response and switches on advertising broadcast.

```

1      void bleSetup() {
2          preferences.begin("pilldisp", false);
3
4          NimBLEDevice::init("PillDispenserESP32");
5          NimBLEServer *pServer = NimBLEDevice::createServer();
6          pServer->setCallbacks(new MyServerCallbacks());
7
8          NimBLEService *pService =
              ↳ pServer->createService(SERVICE_UUID);
9
10         pSetDeviceTimeCharacteristic =
              ↳ pService->createCharacteristic(

```

```

11     CHAR_SET_DEVICE_TIME_UUID, NIMBLE_PROPERTY::WRITE |
        ↳ NIMBLE_PROPERTY::WRITE_NR);
12     pSetDeviceTimeCharacteristic->setCallbacks(new
        ↳ MyCharacteristicCallbacks());
13
14     pGetTimeUntilNextDispenseCharacteristic =
        ↳ pService->createCharacteristic(
15     CHAR_GET_TIME_UNTIL_NEXT_DISPENSE_UUID,
        ↳ NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::NOTIFY);
16     pGetTimeUntilNextDispenseCharacteristic->setValue("Next
        ↳ Dispense: Unknown");
17     . . .
18     // Restore persisted values -----
19     String loadedTime = preferences.getString("devTime", "Device
        ↳ Time: Not Set");
20     pGetDeviceTimeCharacteristic->setValue(loadedTime.c_str());
21
22     String loadedSchedule = preferences.getString("schedule",
        ↳ "Schedule: Empty");
23     pGetDispenseScheduleCharacteristic->setValue(loadedSchedule.c_str());
24
25     String lastDispense = preferences.getString("lastDisp",
        ↳ "Last Dispense: None");
26     pGetLastDispenseInfoCharacteristic->setValue(lastDispense.c_str());
27
28     pService->start();
29
30     NimBLEAdvertising *pAdvertising =
        ↳ NimBLEDevice::getAdvertising();
31     pAdvertising->addServiceUUID(SERVICE_UUID);
32     pAdvertising->setScanResponse(true);
33     NimBLEDevice::startAdvertising();
34 }

```

Listing 9: bleSetup() function responsible for initializing the BLE stack

The final part of the Bluetooth implementation file is the **bleLoop()** function

```

1 void bleLoop() {
2     static unsigned long lastUpdateTime = 0;
3     if (deviceConnected && (millis() - lastUpdateTime > 5000)) {
4         lastUpdateTime = millis();
5         String timeVal = "Next: " + String(millis() / 1000) + "s
            ↳ (simulated)";
6         pGetTimeUntilNextDispenseCharacteristic->setValue(timeVal.c_str());
7         pGetTimeUntilNextDispenseCharacteristic->notify();
8     }
9 }

```

Listing 10: bleLoop() function responsible for initializing the BLE stack

5 Development of a Remote-APP

AI Disclaimer: AI was extensively used to generate code for the App. The file structure, descriptions of main functions and architecture of the app as a whole were not designed by AI, because this information has laid down the groundwork for efficient use of AI to write this said code. Considering this, the chapter will not focus as much on code snippets with in-depth explanation of how the code functions, but rather on a structural overview and architecture of the project. File and app structure, most important functions and state selection will be covered here. The code provided by AI was reviewed and tested for its functionality, however it was not tested for the security (A common concern and weak spot of AI-Generated code), because of lack of such requirement and time constraints. Motivation and methodology for the use of AI will be discussed in the Results Analysis and Discussion section.

Note: For programming part, all the source code is available under authors Github Repository here [14]. Code for this section to be found in the **Android/Dispenser** folder of the repository.

Existence of a remote app is one of the main requirements and a hardest one. The problem here lies in the amount of choices one needs to do upfront, which also contributes to uncertainty about the extent of the project, which will be talked about in Results Section in greater detail. For platform, Android was chosen. Alternatives were Web-based User Interface with Javascript+HTML or IOS. Android was chosen over WebUI or IOS solely from personal preference. A significant amount of time was also spent designing the structure of an app to make it ergonomic, to separate all the functions into different context windows.

The App is structurally subdivided into multiple parts:

1. **Bluetooth connection interface (com.example.dispenser.ble)** package Consists of 2 files: **BluetoothLeManager.kt** and **GattAttributes.kt**.

BluetoothLeManager.kt contains all the functions needed to establish bluetooth communication with the microcontroller using BLE protocol, such as environment scan, dispenser identification, reading and writing data using BLE.

GattAttributes.kt contains all the UUIDs used by the microcontroller with names assigned to them, to avoid magic numbers in the code and replace them with named constants. This is a common practice everywhere, especially in microcontroller programming (where even a value of 0 is often rather depicted as LOW in the code, to avoid confusion)

2. **User Interface (com.example.dispenser.screens)** package consists of 3 files that represent their assigned context windows:

- **DevConnScreen.kt** Contains the code for the UI of the "Device Connection" view. It contains the button to scan the environment for all the bluetooth devices, which are listed in the list under the **Found Devices**. At the very bottom there is also a connection status of the app to the device. When There is also a detection method (based on the device name) to detect the pill dispenser. The app will not attempt to connect to any other device.
- **DevConfScreen.kt** Contains the code for the UI of the "Device Configuration" view. It contains all the settings regarding the configuration of device,

such as: Time of dispense, schedule template load/save, view current schedule, ability to edit or delete current schedule, as well as adding new schedule template and the schedule save/load indicator. It also contains the information about on-board clock and the ability to set or change it

- **DevMonitoringScreen.kt** Contains the code for the UI of the "Monitoring & Status" view. It contains all the miscellaneous information about the device such as: Connection status, Last dispensed time, time until next dispense. The device logs are also available there. Logs contain information about bluetooth connection, information about past dispense events.
 - **com.example.dispenser.screens.devconnection** package contains a class called **DevConnViewModel.kt** that contains functions related to updating the state of the connection view. It provides live update on information sent and received between the Device and the App, using **BluetoothLeManager**.
 - **com.example.dispenser.ui.theme** package defines the application style: colors, fonts, themes etc.
3. **AndroidManifest.xml** Declares information about the App: Permissions, hardware requirements for the device, **MainActivity.kt** as an executive function (similar to main.c in C).

The next section will cover in specific detail the initial requirements given to the AI to generate the code, as well as the results of the code review and how it actually functions.

5.1 Bluetooth Connection Interface

The requirements for bluetooth connection on the side of Android App are as follows:

1. Scan for devices, identify the device that is a Pill Dispenser (using Name as an identifier)
2. Connect to the device using BLE
3. Read Characteristics(GATT Attributes):
 - **CHAR_GET_DEVICE_TIME_UUID** to get the current time set on the dispenser.
 - **CHAR_GET_DISPENSE_SCHEDULE_UUID** to get the currently programmed dispense schedule from the dispenser.
 - **CHAR_GET_LAST_DISPENSE_INFO_UUID** to get information about the most recent dispense event
 - **CHAR_GET_TIME_UNTIL_NEXT_DISPENSE_UUID** to get the remaining time until the next scheduled dispense.
 - **CHAR_GET_DISPENSE_LOG_UUID** retrieve a log of past dispense events.
4. Write characteristics (Also GATT Attributes):
 - **CHAR_SET_DEVICE_TIME_UUID** to send the current time from the App to the dispenser.

- **CHAR_SET_DISPENSE_SCHEDULE_UUID** to send a new dispense schedule (time) to the dispenser.
- **CHAR_TRIGGER_MANUAL_DISPENSE_UUID** to send a command to make the dispenser dispense pills immediately.

5. Manually disconnect from device

All these initial requirements were implemented in code. GATT attributes form an interfacing protocol contract between the mobile application and the microcontroller. This shared definition ensures both systems can correctly interpret the BLE services and characteristics. Consequently, these attributes are mirrored in the microcontroller's firmware, as detailed in Section Microcontroller Programming. As mentioned above, the Bluetooth connection interface consists of 2 files, **BluetoothLeManager.kt** and **GattAttributes.kt**. The first one contains all the functions and classes for the Backend-Frontend interface, while the second one contains GATT attributes and their corresponding UUIDs. Code block 11 is the full **GattAttributes.kt** file. This file contains all the Attributes that were present in the requirements, as well as some service attributes. The standard way of defining it is `val NAME_UUID: UUID = UUID.fromString("UUID")`, where **val** defines a read-only nature of the value (similar, but not quite the same as **const** keyword in C++), **UUID** = `UUID...` is the Java UUID class.

One interesting property of Kotlin as a programming language can be seen in this code. Line 4 contains an import of a Java library, this means that Kotlin is interoperable with Java and can use its libraries.

Moving forward, some properties of the Kotlin language need to be established:

1. **Mutability**: References to immutable objects or variables cannot be changed once created. When modification appears to happen, a new object (with different reference) is created with the modification applied to new object only [17] [18].
2. **Null-safety**: default variable declaration forbids it to have a **null** value i.e. definition strictly follows declaration and memory gets immediately allocated to the variable. **Nullable** (i.e. those which are able to hold **null** value) can be created by appending a '?' symbol e.g.: `var b: String? = null`. It is compile-time safety feature aimed at reduction of runtime Null-Pointer exceptions [19].
3. **Coroutines and Flows** allow multithreading and simultaneous execution of multiple functions. Coroutines Provide means for writing asynchronous, non-blocking code in a sequential style [20]. Flows are asynchronous data streams [21].
4. **StateFlow** is a special type of flow that is often used in the code below. It has 3 important Properties: State-Holder (always contains a value), Observable (all state collector functions receive information about state change of this object) and "Hot Flow" (A flow is hot because its active instance exists independently of the presence of collectors. This is opposed to a regular Flow, which is cold and is started separately for each collector)[22].
5. **Scope Functions** (**let**, **run**, **with**, **apply**, **also**) allow execution of block of code within the context of a given object. Allow referring to an object in different ways and run operations with it as a reference [23].

Listing 11: GATT attributes defined in a separate file

```
1 // GattAttributes.kt
2 package com.example.dispenser.ble
3
4 import java.util.UUID
5
6 object GattAttributes {
7     // Service UUID
8     val SERVICE_UUID: UUID = UUID.fromString("03339647-3f4e-43df-abff-
        ↪ fac54287cf1a")
9
10    // Writable Characteristics
11    val CHAR_SET_DEVICE_TIME_UUID: UUID = UUID.fromString("65232f1d-618
        ↪ a-4268-9050-0548142a4536")
12    val CHAR_SET_DISPENSE_SCHEDULE_UUID: UUID = UUID.fromString("999
        ↪ c584e-06c0-49a1-995a-66b7c802ac1b")
13    val CHAR_TRIGGER_MANUAL_DISPENSE_UUID: UUID = UUID.fromString("36
        ↪ bb95f2-e57e-4db9-b9aa-fb6541ee784e")
14
15    // Readable Characteristics
16    val CHAR_GET_DEVICE_TIME_UUID: UUID = UUID.fromString("272ee276-
        ↪ e37e-4d78-8c5e-bb7225d35074")
17    val CHAR_GET_DISPENSE_SCHEDULE_UUID: UUID = UUID.fromString("
        ↪ b53c2ed4-ae26-476d-8414-011a025dddfc")
18    val CHAR_GET_LAST_DISPENSE_INFO_UUID: UUID = UUID.fromString("40
        ↪ d3b5d8-5480-4b7b-a115-5fe86bf17d7d")
19    val CHAR_GET_TIME_UNTIL_NEXT_DISPENSE_UUID: UUID = UUID.fromString(
        ↪ "4b14acc4-768a-43e1-9d6c-0d97307e2666")
20    val CHAR_GET_DISPENSE_LOG_UUID: UUID = UUID.fromString("6f182da7-
        ↪ c5a8-40ab-a637-f97ed6b5777b")
21
22    // Descriptor for enabling notifications/indications
23    val CLIENT_CHARACTERISTIC_CONFIG_UUID: UUID = UUID.fromString("
        ↪ 00002902-0000-1000-8000-00805f9b34fb")
24 }
```

BluetoothLeManager.kt Contains all the logic of the Android app related to interacting with the device. The code block 12 is the top-level structure of the file. At the highest level there is package definition, library imports, service constant TAG used for logging and main. This file itself is too big (740 lines of code) to do an overview line-by-line and its exact functionality is not the scope of this thesis. In this case, a top level overview of the contents of the file will provide the context for understanding the project as a whole, without digging too deep in technical implementation, specifics of the language and the Platform and their interaction with BLE protocol.

BluetoothLeManager is a class that contains all functions and values for the interface. It relies heavily on built-in Android Package called `android.bluetooth` [24].

The class takes 2 input parameters:

1. **private val context:** `Context` to get the `BluetoothManager` (and subsequently the `BluetoothAdapter`) using `context.getSystemService(Context.BLUETOOTH_SERVICE)` as an entry point to all Bluetooth operations. Consequently, also handles Permissions and GATT connections.
2. **private val coroutineScope:** `CoroutineScope` defines the lifecycle and context for any new coroutines launched within it. Since BLE operations are asynchronous, this handles it by defining a coroutine on the Android side for each operation.

Next, within the class, objects meant for the correct function of BLE are created:

- **`_characteristicUpdate`** is a private(visible only inside the class), mutable active flow. It's used internally by `BluetoothLeManager` to emit events when a characteristic's value is updated.
- **`characteristicUpdate`** is the public(visible outside the class), read-only version of `_characteristicUpdate`. External classes can read flow information from this variable to be notified of characteristic value changes. **Note:** StateFlows are paired like that in the code, in the next instances where public and private instance of the same StateFlow are present, both of them will be described under the same bullet point
- **`discoveredCharacteristics`** is a private mutable map that stores the objects called `BluetoothGattCharacteristic` that are discovered on the connected peripheral. Values received from the Device are written within this variable.
- **`bluetoothManager`** is an instance of the Android Built-in `BluetoothManager` system service[24]. This is the primary entry point for accessing Bluetooth functionality.
- **`bluetoothAdapter`** is the local device's Bluetooth adapter(hardware) object, also from built-in Android package [24]. Nullable in case when a device with this App on it doesn't have Bluetooth. With this all standard Bluetooth operations, such as switching on, scanning and connection can be made.
- **`bleScanner`** is an instance of `BluetoothLeScanner`(a method of built-in Android Bluetooth class) used specifically for scanning for BLE devices. by `lazy` is a keyword indicating that this object(`bleScanner`) will initialize only when its parent (`bluetoothAdapter?.bluetoothLeScanner`) is initialized.

- **scanJob** is a nullable(notice the '?' symbol) Coroutine Job used to manage the lifecycle of an ongoing BLE scan operation. Allows asynchronous scan operation to be started, monitored and cancelled.
- **connectJob** is a nullable Coroutine Job used to manage the lifecycle of an ongoing BLE connection attempt. Allows asynchronous GATT connection.
- **_foundDevices** is private, mutable StateFlow that holds the current list of discovered BLE devices. it gets internally updated by the **leScanCallback** as new devices are found during a scan. While the public version of it Exposes the list of discovered devices to the **DevConnViewModel**, which allows the UI to display found devices.
- **_isScanning** is a private, mutable StateFlow indicating whether a scan is currently active. Used internally to track the scanning state and prevent multiple concurrent scans. Public version of it allows the UI to observe the current scanning status.
- **_connectionStatus** is a private, mutable StateFlow which holds the current connection status (e.g., Disconnected, Connecting, Connected, Error). Used internally to reflect the real-time state of the BLE link. Public version of it allows UI to display the current connection status and to make decisions based on this state (e.g., enabling/disabling certain actions).
- **_connectedDevice** A private, mutable StateFlow, it holds information not only about whether a device is connected, but also about the currently connected device. Public version of it Allows the UI to know which device is currently connected.
- **currentGatt** is a nullable instance of built-in android BluetoothGatt class from the android.bluetooth package[24] that provides GATT client functionality. When a device is not connected, this class can be **null**. It is a main object of all interactions with connected device.

The next 3 functions are made for permission checks:

- **hasPermission** is an utility function to check if a specific permission (passed as a String) has been granted to the application.
- **hasRequiredScanPermissions** Checks if all necessary permissions for BLE scanning are granted. It also contains security features that prevent the App from asking for permissions that might not exist on the device (e.g. when Android version is too old).
- **hasRequiredConnectPermissions** Serves the same purpose as function above, but this time for connection permissions.

The following two functions and one object handle the scanning function of the App:

- **startScan** is a function that initiates a scan to discover all advertising devices in the reach. It performs permission checks, Checks if Bluetooth is enabled, Sets **_isScanning** to true and clears **_foundDevices**, Starts the scan, Launches a **scanJob** to manage the scan, including a timeout (30 seconds) after which the scan is automatically stopped and finally, Updates **_connectionStatus** with an error if prerequisites are not met.

- **stopScan** is a function that Stops an ongoing BLE scan. It Checks if a scan is actually in progress, Calls `bleScanner?.stopScan` using the `leScanCallback`, Sets `_isScanning` to **FALSE**, Cancels the `scanJob` and finally, if there is an error during the stopping process, Handles potential Exceptions during the stop operation.
- **leScanCallback** is an instance of `ScanCallback` class provided by the `bluetooth.le` package (it is different package that is made specifically for BLE connection) [25]. Its methods are invoked by the system when scan events occur. Using this class, device filtering is handled. If scan detects a device called "PillDispenserESP32" it will be marked with the `isPillDispenser` flag, outlining it and putting it at the top of the list of scanned devices.

The next functions handle the connection logic:

- **connectToDevice** is a function that Initiates a GATT connection to a BLE peripheral specified by its MAC address. First, it performs permission and bluetooth availability checks, then retrieves the `BluetoothDevice` object using the provided address, then launches a `connectJob` to handle the asynchronous connection process and finally, calls `deviceToConnect.connectGatt()` with the `gattCallback` to establish the connection.
- **disconnect** is a function that disconnects from the currently connected BLE peripheral and closes the GATT client. First, it checks if `currentGatt` is not null i.e. it is connected, then it performs permission checks, afterwards it calls the command `currentGatt?.disconnect()`, however the actual disconnection and resource cleanup (closing GATT) are typically handled in the `onConnectionStateChange` callback and finally it Cancels the `connectJob` if it was related to an ongoing connection attempt.
- **writeCharacteristic** function is used to write into the writable characteristics on the device. First, it checks for connect permissions and if `currentGatt` is valid. Then, retrieves the `BluetoothGattCharacteristic` from `discoveredCharacteristics`, so that it will know what writable characteristics are on the device. Then, it verifies that the characteristic supports the specified `writeType` (verifies that it is writable), calls `currentGatt?.writeCharacteristic()` and finally, if the write operation has successfully been initiated, it returns **TRUE**.
- **readCharacteristic** is in nature very similar to `writeCharacteristic`, but for read-only characteristics on the device. First, it checks for connect permissions and retrieves `BluetoothGattCharacteristic` (same as previous function), then verifies the read-only property of the characteristic, Calls `currentGatt?.readCharacteristic()` and finally, returns **TRUE** if read operation was initiated successfully.
- **setCharacteristicNotificationEnabled** is a function that enables or disables notifications (or indications) for a given characteristic. Currently implemented, but not used, might be used for debugging and fine-tuning notifications.
- **enableNotificationsFor** is a private helper function specifically called during the `onServicesDiscovered` phase. This is a function that is more simple and robust than the one above. It centralizes the logic for enabling notifications for all relevant "GET_..." characteristics as soon as they are discovered, simplifying the connection setup process

The next object is a key component for the functionality of the whole BLE communication logic.

gattCallback is an instance of built-in Android class called `BluetoothGattCallback`[24] provided to the `connectGatt` method. This is the heart of managing an active BLE connection and its asynchronous events. To accomplish this task, the class has methods built in it:

- **onConnectionStateChange** is called when the connection state changes. If connected successfully, it triggers service discovery(`gatt?.discoverServices()`), on disconnection, it closes the GATT client.
- **onServicesDiscovered** is called after `discoverServices()` completes. If successful, it iterates through the services and characteristics of the connected device. It populates the `discoveredCharacteristics` list with relevant characteristics defined in `GattAttributes` (SET/GET/TRIGGER UUIDs). It also calls `enableNotificationsFor()` function for readable characteristics to automatically subscribe to updates. If the required service or critical characteristics are not found, it updates `_connectionStatus` to an error and disconnects.
- **onCharacteristicRead** is called when a characteristic read operation completes.
- **handleCharacteristicRead** If the read was successful (`GATT_SUCCESS`), it emits the characteristic's UUID and value to the `_characteristicUpdate` flow.
- **onCharacteristicWrite** Called after a characteristic write operation completes. Logs the success or failure. If failed, updates `_connectionStatus` with an error.
- **onCharacteristicChanged** is called when the peripheral sends a notification or indication for a characteristic to which the client has subscribed.
- **handleCharacteristicChanged** is a method that emits the characteristic's UUID and updated value to the `_characteristicUpdate` flow.
- **onDescriptorWrite** is called after a descriptor write operation completes (e.g. to enable/disable notifications). It logs the outcome.

The final function of the class is **cleanup** function. It releases all resources used by the `BluetoothLeManager`. this function is called when the model is no longer needed. it resets all states related to Bluetooth to their default state (disconnected, not scanning).

Listing 12: Top level structure of the `BluetoothLeManager.kt` file

```

1  package com.example.dispenser.ble
2  import . . .
3  private const val TAG = "BluetoothLeManager"
4
5  class BluetoothLeManager(private val context: Context, private val
    ↪ coroutineScope: CoroutineScope)
6  {
7      private val _characteristicUpdate = MutableSharedFlow<Pair<UUID,
    ↪ ByteArray>>()
8      val characteristicUpdate: SharedFlow<Pair<UUID, ByteArray>> =
    ↪ _characteristicUpdate.asSharedFlow()
9  
```

```

10     private val discoveredCharacteristics = mutableMapOf<UUID,
        ↳ BluetoothGattCharacteristic>()
11
12     private val bluetoothManager = context.getSystemService(Context.
        ↳ BLUETOOTH_SERVICE) as BluetoothManager
13     private val bluetoothAdapter: BluetoothAdapter? =
        ↳ bluetoothManager.adapter
14     private val bleScanner by lazy { bluetoothAdapter?.
        ↳ bluetoothLeScanner }
15
16     private var scanJob: Job? = null
17     private var connectJob: Job? = null
18
19     private val _foundDevices = MutableStateFlow<List<
        ↳ UiBluetoothDevice>>(emptyList())
20     val foundDevices: StateFlow<List<UiBluetoothDevice>> =
        ↳ _foundDevices.asStateFlow()
21
22     private val _isScanning = MutableStateFlow(false)
23     val isScanning: StateFlow<Boolean> = _isScanning.asStateFlow()
24
25     private val _connectionStatus = MutableStateFlow<ConnectionStatus>
        ↳ >(ConnectionStatus.Disconnected)
26     val connectionStatus: StateFlow<ConnectionStatus> =
        ↳ _connectionStatus.asStateFlow()
27
28     private val _connectedDevice = MutableStateFlow<UiBluetoothDevice>
        ↳ ?>(null)
29     val connectedDevice: StateFlow<UiBluetoothDevice?> =
        ↳ _connectedDevice.asStateFlow()
30
31     private var currentGatt: BluetoothGatt? = null
32
33     // Helper functions for permission checking
34     private fun hasPermission(permission: String): Boolean
35     private fun hasRequiredScanPermissions(): Boolean
36     private fun hasRequiredConnectPermissions(): Boolean
37
38     // Scan logic
39     fun startScan()
40     fun stopScan()
41     private val leScanCallback = object : ScanCallback { ... }
42
43     // Connection logic
44     fun connectToDevice(deviceAddress: String)
45     fun disconnect()
46
47     // Characteristic operations
48     fun writeCharacteristic(characteristicUUID: UUID, value:
        ↳ ByteArray, writeType: Int = ...): Boolean
49     fun readCharacteristic(characteristicUUID: UUID): Boolean
50     fun setCharacteristicNotificationEnabled(characteristicUUID: UUID
        ↳ , enable: Boolean): Boolean
51
52     // Notification helper
53     private fun enableNotificationsFor(characteristic:
        ↳ BluetoothGattCharacteristic)
54

```

```
55     // GATT callback
56     private val gattCallback = object : BluetoothGattCallback { ... }
57
58     // Cleanup
59     fun cleanup()
60
61 }
```

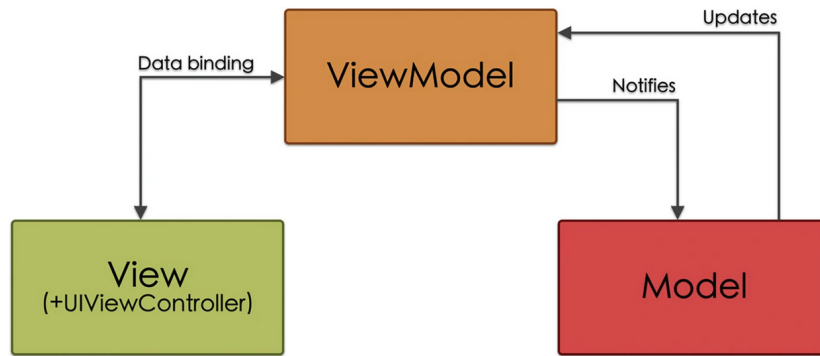


Figure 18: Model-View-ViewModel Structure. This image explains what model consists of and how different parts of model interact with each other. Credit [26]

5.2 User Interface

Having finished with describing how the App handles interface between microcontroller using BLE, the Graphical User Interface (GUI) of the app will be covered next. It consists of 3 main views (called **Device Connection, Device Configuration and Monitoring & Status**), each of them contains different information and functions. Switching between views is done via a drawer panel on the left-hand side of the app, which slides open when the button at the top left is pressed.

Once again, before moving forward, there is a certain terminology that needs to be covered. This thesis uses terminology used by official design guide[27] from **Android Developers** team, which is an industry standard.

- **Jetpack Compose** while not a technical term as the others are on the list, Jetpack Compose is the modern toolkit for building native Android User Interface (UI). It defines the paradigm under which all the other terms operate. It operates under the Model–view–viewmodel (MVVM) design pattern.
- **MVVM** is an architectural design pattern that separates an application into three (View, Model, ViewModel) components to enhance modularity, testability, and maintainability. Advantages of this pattern is that it separates logic from presentation, allowing easier testing, debugging and troubleshooting[26].
- **Model** is responsible for manipulation, access and storage of data.
- **View** is the part of the architecture that a user sees on the screen.
- **ViewModel** class is a business logic or screen level state holder. ViewModel is the core of the MVVM architecture and sits between the View and the Model. Its principal advantage is that it caches state and persists it through configuration changes. The alternative to a ViewModel is a plain class that holds the data displayed in the UI. This can become a problem when navigating between activities or Navigation destinations. Doing so destroys that data if it isn't stored using the saving instance state mechanism.
- **ViewModelFactory** is a class responsible for creating instances of ViewModels. It's needed because ViewModels often have dependencies that need to be provided to them when they are created.

- **UI State** represents what to show on the screen. For the Device Connection screen, this includes whether the app is currently scanning, the list of found Bluetooth devices, the current connection status (e.g., "Disconnected", "Connecting", "Connected"), and any error messages. This state typically lives in or is managed by the **ViewModel**.
- **Composable Function (or "Composable")** This is the fundamental building block of UI in Jetpack Compose. Instead of traditional XML layouts, UI is defined by writing functions annotated with `@Composable`. These functions describe what the UI should look like for a given state, not how to construct it step-by-step.
- **Recomposition** is the process of Jetpack Compose re-running Composable functions when their input **State** changes. If the `scanStatus` (part of the UI State) changes from "Disconnected" to "Scanning", Compose will re-run (recompose) only the parts of UI that depend on `scanStatus`, updating them automatically. UI elements don't need to be manually updated.
- **Layout Composables (Column, Row, Box)** are special Composables used to arrange other Composables on the screen. `Column` arranges its children vertically, `Row` horizontally, and `Box` allows stacking elements or positioning them relative to its edges.
- **Event Handling (Callbacks/Lambdas)** is based on user interaction with UI. Composables can trigger actions. This is usually done by passing lambda functions (callbacks) as parameters to the Composables.
- **StateFlow** has already been defined in the Bluetooth Connection Section. It is a state-holder observable "hot" flow that always has a value and emits updates to its collectors. This term is even more important here, as it creates a bridge for understanding how User Interface interacts with data. In context of User Interface, it is a primary mechanism for the **ViewModel** to expose **UI State** to the Composable. Composable functions can observe (or "collect") these `StateFlows`. When the `StateFlow` emits a new value, `collectAsState()` receives this update and signals to the Compose runtime that the state read by a Composable has changed.

5.2.1 MainActivity.kt

The Main entry file to the whole app is called **MainActivity.kt**. The fundamental part of the UI is connected through this file as can be seen on image 19. It is worth reviewing it here before delving deeper into the each view. The top level structure of the code can be seen in code block here 13.

Main application structure is defined in the **AppScaffoldWithDrawer** composable function. It contains top app bar, a navigation drawer, and the main content area where different views are displayed. This function is called within the **MainActivity** class's `onCreate` method to set up the UI. **AppScaffoldWithDrawer** has numerous tasks:

1. **Navigation:** Manages the `ModalNavigationDrawer` to allows switching between different sections of the app ("Device Connection", "Device Configuration", "Monitoring & Status").

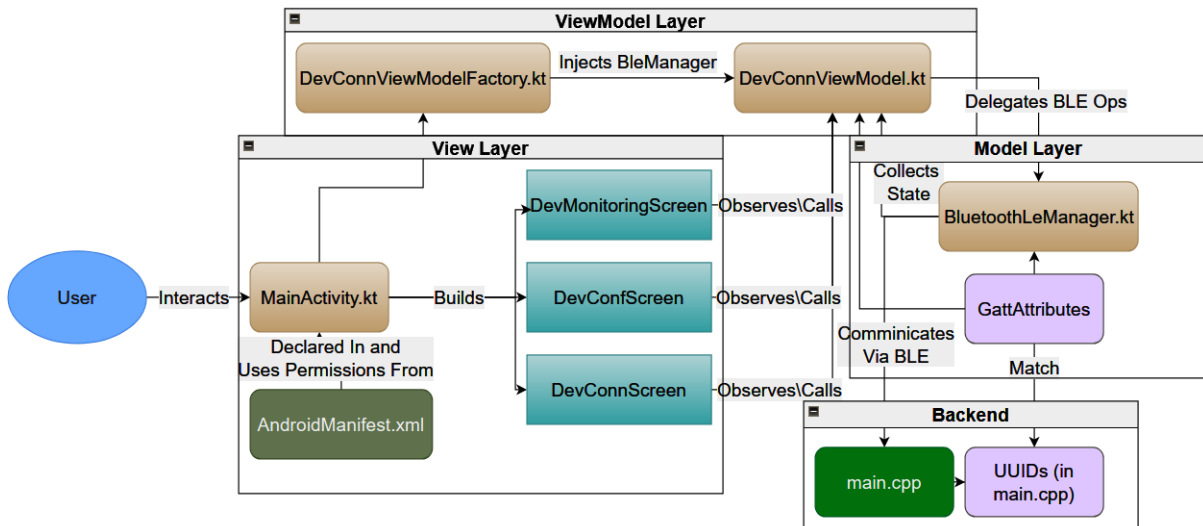


Figure 19: Diagram of an Android App showing how its different components interact with each other and with backend. The structure is divided into 3 parts: View Layer, ViewModel Layer and Model Layer, corresponding to MVVM paradigm.

2. **Screen Switching:** Uses a `when` (similar to `case` and `switch` commands in C/C++) statement to display the appropriate screen in the Scaffold's content area.
3. **ViewModel Access:** It receives the `devConnViewModelFactory` and uses it to obtain an instance of `DevConnViewModel`. This `ViewModel` is then passed down to the relevant screens.
4. **Conditional UI:** It conditionally renders content within the "Device Connection" screen path (e.g. Popups that prompt granting permissions and enabling Bluetooth).
5. **Callbacks:** Passes down callbacks to `MissingPermissionsScreen` and `BluetoothDisabledScreen`.

Besides `AppScaffoldWithDrawer` function, there are also various supplementary functions:

- **MissingPermissionsScreen** is a simple UI screen displayed when the necessary Bluetooth permissions have not been granted by the user. It provides a "Grant Permissions" button that triggers the `onRequestPermissions` callback (which is implemented in `MainActivity`) to launch the permission request.
- **BluetoothDisabledScreen** is similar to `MissingPermissionsScreen`, but for when Bluetooth is disabled. Also provides an "Enable Bluetooth" button to switch it on.
- **DeviceConfigurationScreen** This is a legacy device configuration view, left from older iterations of the code. Currently not used, still left because it might be used for debugging. Device configuration view is currently handled in a separate file `DevConfScreen.kt`
- **MonitoringStatusScreen** Same as above, legacy view for Device Monitoring & Status screen, currently not used. This View is currently handled by `DevMonitoringScreen.kt` file.

- **AppScaffoldWithDrawerPreview** is an assistance function for Prototyping. Android Studio has built-in app preview functionality, this helper function allows seeing how **AppScaffoldWithDrawer** looks without running the entire app on an emulator or device.

Next, **MainActivity** class ought to be reviewed next. It is a primary **Activity** of the application, handles lifecycle, Compose UI and manages permissions and Bluetooth initialization. It has 3 parameters:

- **private val applicationScope** is a **CoroutineScope** that lives as long as **MainActivity** is explicitly active. It consists of 2 components: **SupervisorJob** ensures that if one child coroutine fails, others are not cancelled. **Dispatchers.Default** is used for CPU-intensive work off the main thread.
- **private lateinit var bluetoothLeManager** holds the instance of **BluetoothLeManager**, the class that was extensively covered in the Bluetooth Connection Section.
- **private val blePermissions** is used to define an array of Bluetooth-related permissions required by the app, adapting to different Android versions (API 31/Android S and above have different permission requirements). Necessary for compatibility with older devices that have a different set of permissions.

Then, it has 4 different methods, each serving different purpose:

- **onCreate** is a Lifecycle method, which is called when activity is starting. It fulfills many important tasks: Initializes the **BluetoothLeManager**. Using **setContent { ... }** it creates entry point for Jetpack Compose, which defines the UI content. Besides that, it also sets the stage for the function of the app as a whole: it establishes **Permission Handling**, **Bluetooth Enabled Checking**, **Initial Permission/Bluetooth Requests**, **ViewModel Factory Creation** and finally **Main UI Setup**.
- **checkAllPermissionsGranted** is a private helper method that serves as a checklist to go through and check if all permissions (defined in **blePermissions**) have been granted. It initializes **hasPermissions** on creation from method above and can be called again to re-check permissions.
- **isBluetoothAdapterEnabled** is a private helper method that checks if Bluetooth adapter is currently enabled. It initializes **isBluetoothEnabled** in **onCreate** and by **enableBluetoothLauncher** to check if Bluetooth is enabled.
- **onDestroy** is a lifecycle method for termination of the task that is used to release resources. It works together with the **bluetoothLeManager.cleanup()** method from the **BluetoothLeManager** class to clean up the resources.

Listing 13: Top-level structure of the **MainActivity.kt** file

```

1 package com.example.dispenser
2 import . . .
3
4 class MainActivity : ComponentActivity() {
5
```

```

6     private val applicationScope = CoroutineScope(/* SupervisorJob()
    ↪ + Dispatchers.Default */)
7     private lateinit var bluetoothLeManager: BluetoothLeManager    //
    ↪ Instantiate early
8
9     // List of permissions
10    private val blePermissions =
11    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) { ... } else
        ↪ {
12        arrayOf(
13            Manifest.permission.BLUETOOTH,
14            Manifest.permission.BLUETOOTH_ADMIN,
15            Manifest.permission.ACCESS_FINE_LOCATION    // or coarse
16        )
17    }
18
19    // Lifecycle
20    override fun onCreate(savedInstanceState: Bundle?) { ... }
21    private fun checkAllPermissionsGranted(): Boolean
22    private fun isBluetoothAdapterEnabled(): Boolean
23    override fun onDestroy() { ... }
24 }
25
26 /* ----- Composables ----- */
27
28 @OptIn(ExperimentalMaterial3Api::class)
29 @Composable
30 fun AppScaffoldWithDrawer(
31     modifier: Modifier = Modifier,
32     permissionsGranted: Boolean,
33     isBluetoothEnabled: Boolean,
34     onRequestPermissions: () -> Unit,
35     onRequestEnableBluetooth: () -> Unit,
36     devConnViewModelFactory: DevConnViewModelFactory
37 ) { ... }
38
39 @Composable
40 fun MissingPermissionsScreen(onRequestPermissions: () -> Unit) {
    ↪ ... }
41
42 @Composable
43 fun BluetoothDisabledScreen(onRequestEnableBluetooth: () -> Unit) {
    ↪ ... }
44
45 // Place-holder composables for future screens
46 @Composable
47 fun DeviceConfigurationScreen() { ... }
48
49 @Composable
50 fun MonitoringStatusScreen() { ... }
51
52 /* ----- Preview ----- */
53 @Preview(showBackground = true)
54 @Composable
55 fun AppScaffoldWithDrawerPreview() { ... }

```

5.2.2 DevConnViewModel.kt

DevConnViewModel.kt is responsible for State management. Following the MVVM pattern it is the ViewModel part of the app. Besides the Factory, this is the sole ViewModel used for this app, as shown here 19. It contains:

- **DevConnViewModel** class is a ViewModel designed to manage the UI-related data for the device connection screen. This ViewModel is used by all views that interact with the connected BLE device. It inherits from Android built-in **AndroidViewModel**, which provides it with an Application context, useful for system services or application-wide resources.
- **scanStatus** is a StateFlow exposes the current status of the BLE scanning process to the UI.
- **foundDevices** is a StateFlow that exposes the current status of the connection to a BLE device.
- **connectingDevice** is a Stateflow that covers the specific device that is currently in the process of connecting or is already connected. It is nullable, therefore it can also be **null** if no device is connected.
- **__lastError** is a private, mutable state holder for any general error messages that the ViewModel itself might generate. Nullable, and is **null** when initialized.
- **displayError** is a StateFlow that Provides a single, consolidated error message for the UI.
- **startScan()**, **stopScan()**, **connectToDevice(device: UiBluetoothDevice)** and **disconnect()** are functions managing interaction with the device. They use corresponding methods from the **BluetoothLeManager** class.
- **setGeneralError** is a helper function to allow setting a custom error message to **__lastError**

The next StateFlows within the **DevConnViewModel** class are related to interaction with Timer/Counter of the device, which plays a pivotal role in its configuration and interaction in general

- **__deviceTime** is a StateFlow that holds the current time of the connected device, as a string. **deviceTime** is a similar non-mutable StateFlow to display the time in UI.
- **__dispenseSchedule** is a StateFlow that contains the current dispense schedule of the connected dispenser device, as a string. **dispenseSchedule** is an immutable variant of this.
- **__lastDispenseInfo** Holds information about the last dispense event from the dispenser. **lastDispenseInfo** is an immutable variant.
- **__timeUntilNextDispense** Holds the calculated or received time until the next scheduled dispense. **timeUntilNextDispense** is an immutable variant.

- **__dispenseLog** Holds a log of dispense events from the dispenser. **dispenseLog** is an immutable variant.

The next code block **init { . . . }** is executed when an instance of **DevConnViewModel** is created. It is used to set up long-lived operations or observers. It launches a coroutine in **viewModelScope** to collect connection data from **bluetoothLeManager.connectionStatus**. If the status is not **Connected**, it resets all the device-specific data (**_deviceTime**, **_dispenseSchedule**, etc.) to their "N/A (Disconnected)" states.

After this block, functions related to sending data from the App to the device are defined:

- **triggerManualDispense** sends a command to the connected BLE device to perform a manual dispense.
- **setDeviceTime** sends a command to the connected BLE device to set its internal time.
- **setDispenseSchedule** sends a command to the connected BLE device to set its dispense schedule.
- **requestDeviceTime** sends a request to the connected BLE device to read its current time characteristic.
- **requestDispenseSchedule** sends a request to read the dispense schedule characteristic.
- **requestLastDispenseInfo** sends a request to read the last dispense info characteristic.
- **onCleared** This method is called when the ViewModel is no longer used and is about to be destroyed. It's the place to clean up any resources tied to the ViewModel's lifecycle.

Listing 14: Top-level structure of the **DevConnViewModel.kt** file

```

1  package com.example.dispenser.screens.devconnection
2
3  import ...
4
5  class DevConnViewModel(
6      application: Application,
7      private val bluetoothLeManager: BluetoothLeManager
8  ) : AndroidViewModel(application) {
9
10     // State for scan status (now reflects manager's scanning state)
11     val scanStatus: StateFlow<ConnectionStatus> = bluetoothLeManager.
        ↳ isScanning
12     .map { isScanning ->
13         if (isScanning) ConnectionStatus.Scanning else ConnectionStatus
        ↳ .Disconnected
14     }
15     .stateIn(viewModelScope, SharingStarted.Lazily, ConnectionStatus.
        ↳ Disconnected)
16
17     // State for found devices (directly from manager)

```

```

18     val foundDevices: StateFlow<List<UiBluetoothDevice>> =
19         ↳ bluetoothLeManager.foundDevices
20
21     // State for device connection status (directly from manager)
22     val connectionStatus: StateFlow<ConnectionStatus> =
23         ↳ bluetoothLeManager.connectionStatus
24
25     // State for the device currently being connected to/connected (
26     ↳ directly from manager)
27     val connectingDevice: StateFlow<UiBluetoothDevice?> =
28         ↳ bluetoothLeManager.connectedDevice
29
30     // State for any last error message (can be a combination or
31     ↳ specific errors)
32     // For now, let's use the connectionStatus error or a general one
33     ↳ .
34     private val _lastError = MutableStateFlow<String?>(null) // For
35     ↳ other types of errors if needed
36     val lastError: StateFlow<String?> = _lastError.asStateFlow()
37
38     // Combine scan errors and connection errors into a displayable
39     ↳ error
40     val displayError: StateFlow<String?> = combine(
41         bluetoothLeManager.connectionStatus, // For connection/scan
42         ↳ errors from manager
43         _lastError // For other general errors set by ViewModel
44     ) { connStatus, generalError ->
45         if (connStatus is ConnectionStatus.Error) {
46             connStatus.message
47         } else {
48             generalError
49         }
50     }.stateIn(viewModelScope, SharingStarted.Eagerly, null)
51
52     fun startScan() { ... }
53     fun stopScan() { ... }
54     fun connectToDevice(device: UiBluetoothDevice) { ... }
55     fun disconnect() { ... }
56     fun setGeneralError(message: String?) { ... }
57
58     private val _deviceTime = MutableStateFlow<String?>("N/A")
59     val deviceTime: StateFlow<String?> = _deviceTime.asStateFlow()
60
61     private val _dispenseSchedule = MutableStateFlow<String?>("N/A")
62     val dispenseSchedule: StateFlow<String?> = _dispenseSchedule.
63         ↳ asStateFlow()
64
65     private val _lastDispenseInfo = MutableStateFlow<String?>("N/A")
66     val lastDispenseInfo: StateFlow<String?> = _lastDispenseInfo.
67         ↳ asStateFlow()
68
69     private val _timeUntilNextDispense = MutableStateFlow<String?>("N
70     ↳ /A")
71     val timeUntilNextDispense: StateFlow<String?> =
72         ↳ _timeUntilNextDispense.asStateFlow()
73
74     private val _dispenseLog = MutableStateFlow<String?>("N/A")
75     val dispenseLog: StateFlow<String?> = _dispenseLog.asStateFlow()

```

```
63     init { ... }
64     fun triggerManualDispense() { ... }
65     fun setDeviceTime(timeString: String) { ... }
66     fun setDispenseSchedule(scheduleString: String) { ... }
67     fun requestDeviceTime() { ... }
68     fun requestDispenseSchedule() { ... }
69     fun requestLastDispenseInfo() { ... }
70     override fun onCleared() { ... }
71 }
```

DevConnViewModelFactory.kt file contains contains a single *DevConnViewModelFactory* class. It is a custom factory responsible for creating instances of **DevConnViewModel**. This is necessary because **DevConnViewModel** requires arguments in its constructor (**application** and **bluetoothLeManager**), which the default ViewModel provider mechanism cannot supply.

5.2.3 Device Connection

The structure behind **Device Connection** view consist of multiple files that contribute to this view:

1. **DevConnScreen.kt** file contains the Composable function **DevConnScreen**, which defines the visual layout and user interface elements for the device connection screen. This file represents the **View** layer of MVVM pattern.
2. **DevConnViewModel.kt** is already covered before. This file contains State management logic. It acts as a bridge between the UI (**DevConnScreen**) and the data layer (**BluetoothLeManager**). It holds the state displayed by the UI and processes user actions. This file represents the **ViewModel** layer of MVVM pattern.
3. **BluetoothLeManager.kt** Covered in previous section, is a class responsible for handling BLE interactions between Device, Android Bluetooth Application Programming Interface (API) and GUI. Since it manages logic and background functionality, it is part of **Model** layer of MVVM pattern.
4. **MainActivity.kt** is the main entry point of this app. It sets up the overall UI structure, including the navigation drawer and the content area where all the screens are displayed.
5. **DevConnViewModelFactory.kt** is a standard ViewModel factory responsible for creating instances of **DevConnViewModel**
6. **GattAttributes.kt** Defines the unique UUIDs for BLE service and characteristics on the ESP32. Already covered in the Bluetooth Connection Section.

Many of these files (**BluetoothLeManager.kt**, **MainActivity.kt** and **GattAttributes.kt**) have already been previously covered and 3 files that we will cover next, to understand how the view is created. **DevConnScreen.kt** file is responsible for layout and UI elements. Its top level structure can be seen in code block 14 It contains:

- **UiBluetoothDevice** data class serves as a simplified representation of a Bluetooth device, made to be displayed in the UI. It's more convenient to use this in Composable functions than the raw Android **BluetoothDevice** object directly. Instances of this class are used to populate the list of found devices displayed on the screen(MAC address, Device name and whether it is a Pill Dispenser).
- **ConnectionStatus** class defines a set of distinct states representing the current status of the Bluetooth scanning and connection process. It is a **sealed** class, which means that all direct subclasses of a sealed class must be declared in the same file as the sealed class itself. Here it is used for a state-switch (using an already known command **when**). It has 4 state objects: **Disconnected**, **Scanning**, **Connecting**, **Connected**, each representing current relationship of an App to the device.
- **DevConnScreen** is a primary composable function that defines the entire layout and behavior of the **Device Connection** screen. It is quite vast function in a sense that through it the whole view is displayed, therefore it is worth documenting it deeper. **DevConnScreen** takes many input parameters:
 - **scanStatus** is the current status of the scanning process

- **foundDevices** is list of discovered Bluetooth devices.
- **connectionStatus** is current status of the device connection.
- **connectingDevice** is the device that is currently being connected to, or is already connected.
- **lastError** general error message that might not be part of either **scanStatus** or **connectionStatus**
- **onStartScanClick** is a callback function invoked when the "Start Scan" button is clicked.
- **onStopScanClick** is a callback function invoked when the "Stop Scan" button is clicked.
- **onDeviceClick** is a callback function invoked when a device in the list is clicked.
- **onDisconnectClick** is a callback function invoked when the "Disconnect" button is clicked.

In essence, this function is responsible for every element of the UI seen within the "Device Connection Screen" window.

- **DeviceListItem** is a helper Composable function responsible for rendering a single item in the list of found Bluetooth devices. In essence, this function is responsible for the contents of a single card from the list of scanned devices. It is called each time a new device is found to draw a card with information about this device within it.
- **DevConnScreenPreview_*** are multiple functions with similar names and purpose. Each of them serves the debugging previewing, allowing checking different states of this window within Android Studio without having to compile and emulate the whole app.

5.2.4 Device Configuration

The Device Configuration screen has the following functionality, as also seen on image 20:

- **View Current Device Time:** Displays the time currently set on the ESP32.
- **Set Device Time:** Manually enter a time string or set the ESP32's time to the phone's current time.
- **View Current Dispense Schedule:** Displays the dispense schedule set on the microcontroller.
- **Set Dispense Schedule:** Enter a new dispense schedule string.
- **Trigger Manual Dispense:** Command the ESP32 to perform a single dispense operation.

All these interactions need an active Bluetooth connection to the device. This view depends on these GATT attributes to function:

Read Characteristics:

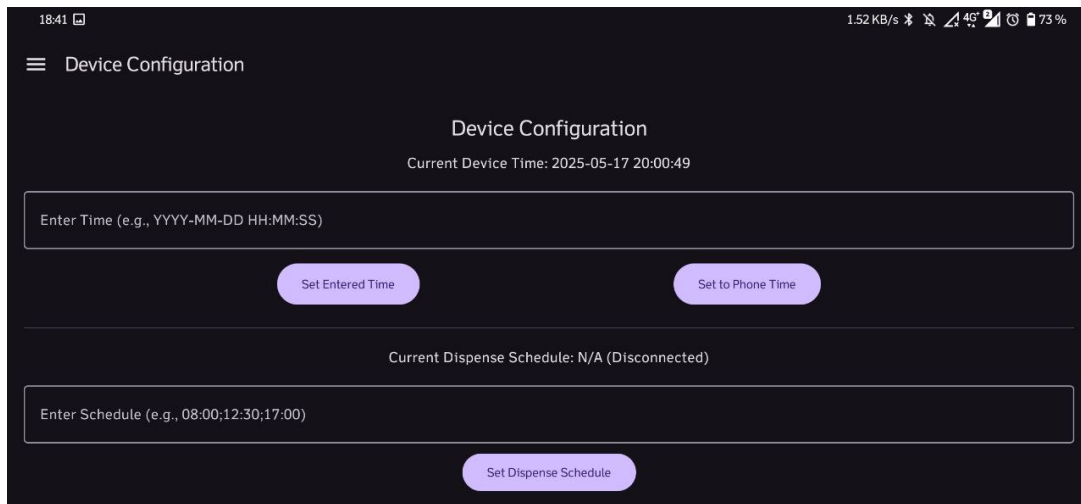


Figure 20: Layout of the device configuration view file

- **GattAttributes.CHAR_GET_DEVICE_TIME_UUID** to display the "Current Device Time."
- **GattAttributes.CHAR_GET_DISPENSE_SCHEDULE_UUID** to display the "Current Dispense Schedule."

Write Characteristics

- **GattAttributes.CHAR_SET_DEVICE_TIME_UUID** to send the new time string to the ESP32.
- **GattAttributes.CHAR_SET_DISPENSE_SCHEDULE_UUID** to send the new schedule string to the ESP32.
- **GattAttributes.CHAR_TRIGGER_MANUAL_DISPENSE_UUID** to command the ESP32 to dispense.

The files contributing to this view are **DevConfScreen.kt**, **DevConnViewModel.kt**, **BluetoothLeManager.kt**, **GattAttributes.kt**, **MainActivity.kt**. They serve the same purpose for this view as for the previous one. What differs the two views is the controller file, in this case **DevConfScreen.kt**. It is the file that defines the structure and functionality of this screen. Unlike **DevConnScreen** file, this one is much smaller, because **DevConnScreen** also serves as an initialiser for services, permissions etc. which is not present here. Essentially all the functionality is relegated to other files, classes and functions. This file only binds them together and lays down the structure of the UI. as seen in block 15, This file contains a sole function called **DeviceConfigurationScreen** which takes **DevConnViewModel** ViewModel as an input parameter to fetch necessary data from the device.

Listing 15: Top-level structure of the **DeviceConfigurationScreen.kt** file

```

1 package com.example.dispenser.screens
2
3 import ...
4
5
```

```

6  @OptIn(ExperimentalMaterial3Api::class)
7  @Composable
8  fun DeviceConfigurationScreen(devConnViewModel: DevConnViewModel) {
9      val connectionStatus by devConnViewModel.connectionStatus.
10         ↪ collectAsState()
11
12     val isConnected = connectionStatus == ConnectionStatus.Connected
13
14     var timeToSetText by remember { mutableStateOf("") }
15     var scheduleToSetText by remember { mutableStateOf("") }
16
17     val currentDeviceTime by devConnViewModel.deviceTime.
18         ↪ collectAsState()
19     val currentSchedule by devConnViewModel.dispenseSchedule.
20         ↪ collectAsState()
21
22     LaunchedEffect(isConnected) {
23         if (isConnected) {
24             devConnViewModel.requestDeviceTime()
25             devConnViewModel.requestDispenseSchedule()
26         }
27     }
28
29     Column(
30         modifier = Modifier
31         .fillMaxSize()
32         .padding(16.dp),
33         horizontalAlignment = Alignment.CenterHorizontally,
34         verticalArrangement = Arrangement.spacedBy(10.dp)
35     ) { \UI Components here\ }
36 }

```

5.2.5 Monitoring & Status

The **MonitoringStatusScreen** is designed to provide a read-only view of various operational data and logs from the connected pill dispenser. It allows the user to:

- **View Current Device Time**
- **View Current Dispense Schedule**
- **View Last Dispense Information** Displays details about the most recent dispense event
- **View Time Until Next Dispense** Shows a countdown or status related to the next scheduled dispense (this is simulated/periodically updated by the ESP32 in main.cpp file on the Microcontroller side).
- **View Dispense Log** Shows a raw log of dispense events (currently not implemented, but was designed during project development phase)
- **Refresh Data** A button allows manually refreshing the "Last Dispense Info,"

This screen relies heavily on the ESP32 sending data (Timer/Counter information) via BLE notifications. There are once again multiple files contributing to the functionality of this screen:

- **DevMonitroingScreen.kt** defines the user interface for displaying monitoring information. Breaking the pattern, the main composable function is not the same as the name of the file, is called **MonitoringStatusScreen**. This is a minor oversight, which does not affect functionality of the code. It is the **View** layer of the MVVM paradigm.
- **DevConnViewModel.kt** like in other screens, it manages UI-related data for the monitoring screen, interacts with **BluetoothLeManager** to get this data, and exposes it as observable flows. It is the **ViewModel** layer of the MVVM paradigm.
- **BluetoothLeManager.kt** here too, handles all BLE communication. It is the **Model** layer of MVVM paradigm.

For this screen to function, this View receives following GATT characteristics:

- **GattAttributes.CHAR__GET__DEVICE__TIME__UUID** to display "Device Time"
- **GattAttributes.CHAR__GET__DISPENSE__SCHEDULE__UUID** to display "Dispense Schedule".
- **GattAttributes.CHAR__GET__LAST__DISPENSE__INFO__UUID** to display "Last Dispense Info".
- **GattAttributes.CHAR__GET__TIME__UNTIL__NEXT__DISPENSE__UUID** to display "Time Until Next Dispense".
- **GattAttributes.CHAR__GET__DISPENSE__LOG__UUID** to display "Dispense Log (Raw)".

6 Results Analysis and Discussion

This section provides an analysis of the project’s outcomes and a critical discussion of the development process. The project involved a wide range of tasks spanning multiple technical domains. While valuable experience was gained and certain components were progressed, such as 3D printed body, an existing prototype of an Android app and some backend code, the overall integration and functionality targeted by the initial objectives were not fully achieved. Consequently, the final result is evaluated as not meeting the pre-defined success criteria. The success criterion was this: A fully functioning Pill Dispenser with a remote app to remotely control it. This discussion will explore the contributing factors, focusing on challenges encountered in project management and technical execution, reflect on the lessons learned, and propose directions for future work. Contextualizing these challenges is crucial for understanding the project’s trajectory and final state.

6.1 Challenges Encountered

The difficulties faced during the project can be broadly categorized into two interconnected areas: **Project Management and Execution**, and **Technical Complexity and Resource Constraints**.

6.1.1 Project Management and Execution

Effective project management proved to be a significant challenge, stemming primarily from a lack of prior experience in managing long-term, multi-faceted technical projects.

1. **Planning and Structuring** Early momentum was shaped by *over-confidence that past short, ad-hoc jobs had prepared the author for end-to-end project ownership*. . . An initial underestimation of the need for rigorous, upfront planning led to difficulties. Specifically, the project lacked a sufficiently detailed breakdown of the overall task into manageable sub-components with clear milestones and dependencies. This absence of a granular structure hindered effective progress tracking and adaptation throughout the development cycle.
2. **Resource Availability** Ambiguity regarding the scope of permissible resources (e.g., access to university facilities, specific software licenses, or hardware) and their integration into the project plan was an early challenge. This lack of explicit definition led to a predominantly Do-It-Yourself approach to problem-solving, potentially precluding the use of more optimized or readily available solutions typical in industrial or well-resourced academic environments. This highlighted the necessity for proactive clarification of all available resources at the project’s outset.
3. **Development Methodology and Feedback Loop**: An Agile-inspired iterative approach was initially assumed, anticipating regular feedback to guide development. However, the conditions **were not clearly defined** and there was some confusion about what role each participant (including me) would play. There was an absence of a clearly defined counterpart or stakeholder role dedicated to providing consistent, actionable feedback. This ambiguity made it challenging to effectively solicit, interpret, and integrate feedback, occasionally impeding development velocity. Furthermore, the lack of clearly delineated responsibilities within the project

(as it was executed solely by the author) made whatever feedback was given feel less integrated into a structured workflow.

4. **Task Management and Cognitive Load:** Executing all project tasks single-handedly – from conceptualization and design to implementation and testing across different domains – resulted in significant challenges related to **divided attention**. Constantly shifting between diverse tasks (e.g., 3D modeling, embedded programming, mobile development) proved cognitively demanding and potentially reduced efficiency and focus in each area. This experience highlighted a key learning: the critical importance of comprehensive documentation. Well-structured documentation serves as an essential tool for defining components, standardizing terminology, and maintaining a clear overview, thereby facilitating better intrapersonal communication (tracking one’s own work across domains) and enabling potential future collaboration.

6.1.2 Technical Complexity and Resource Constraints

The project’s ambitious scope required integrating expertise from several distinct technical fields, which posed considerable problems for a single developer.

1. **Scope and Expertise Mismatch:** The project necessitated skills in 3D modeling and printing, Android application development, and microcontroller programming with Bluetooth Low Energy (BLE) communication. While 3D design and printing were navigated successfully, providing valuable practical insights, they consumed a significant portion of the available time.
2. **Software Development Learning Curve:** The software development aspects presented steep learning curves:
 - (a) **Android Development:** Required learning not only Kotlin syntax but also the OOP paradigm as applied to GUI development, Android-specific API, the application lifecycle, build toolchains (e.g., Gradle), and handling platform diversity (API levels, device compatibility). This differs substantially from procedural or scripting languages like C, Python, or MATLAB, particularly concerning memory management, data type handling, and runtime environments.
 - (b) **Microcontroller Programming & BLE:** Required dedicated research into embedded C/C++, microcontroller architecture (referencing datasheets), and the complexities of the BLE protocol stack and its implementation on the target hardware. While manufacturer documentation and libraries provide support, integrating these components effectively remained time-intensive.
 - (c) **Effort Underestimation:** The cumulative effort required to gain proficiency and implement solutions across these diverse domains was initially underestimated. The need to simultaneously learn and apply knowledge in unfamiliar areas significantly impacted the development timeline and the ability to achieve the desired level of integration and polish for the final system. Tackling these complex, disparate technical challenges single-handedly proved to be a major constraint.

6.2 Reflection and Lessons Learned

Despite the project not fully meeting its objectives, the process yielded significant learning outcomes. The development itself delivered a big amount of various domain knowledge in areas of 3D-Design and Printing, Android and Microcontroller development and Project Management:

1. **3D-Design and Printing:** Before starting the project, my experience designing a functional device was limited. Moreover, the designed components would then have to be 3D-printed, which meant that material properties of the filament would have to be taken into account, leading to research and adjustment of the design to facilitate the newly obtained knowledge.
2. **Android Development:** Through this project a lot of insights has been gained on how the Android development environment works. Since I had no experience in programming any User interface harder than basic Command-Line interface, I had to learn how to do it and also using OOP for that. Android platform has also added complexity, since one would need to learn how Application Packaging, Permissions, Device Compatibility and many other components interact together in an Android Application.
3. **Microcontroller Development** I have already had experience developing on an another microcontroller, which was also rather lower-level. Arduino and Platform.io development environments have made it easier to write a backend, however writing a BLE was new for me.
4. **Project Management** This project has had a profound impact on my view of the role and importance of Project Management. It is important to not also divide the tasks in a project-structure plan (which has-been done), but also to clarify the participants, their roles and tasks. Since this hasn't been done initially it lead to difficulties estimating necessary amount of effort to complete the project successfully. It is also important to know in advance (and have it written down somewhere) which resources are made available for the project. There has been no requirements on budget laid down. Generalizing, the high amount of requirements is good because it reduces amount of creative freedom. There is a balancing act at play where if the requirements are too strict, there is only one solution or none at all, while on the other hand, if the requirements are too lax, the creative freedom is too wide which leads to kind of analysis paralysis, in which the developer would have to either lay down requirements themselves (effectively delegating the task of creating a Technical Task onto the developer) or operate on assumptions (which would then require a feedback from the client, whether these assumptions are adequate).

6.3 Next steps

A lot of work remains to be done. From the side of components it might be worth looking into a stronger stepper motor, as the current one has very narrow margin of error and its performance is highly dependent on battery level. What this means is that when the battery level is low, it might not be able to pull the load of 2 mills and extra weight. The construction of the device itself is mostly fine, although the chamber wheels are a bit loose to reduce amount of contact area and friction. If there is a change in components,

it would most certainly also require redesigning the lower deck to fit the new components, as the more powerful stepper motor would also require different (perhaps also bigger) driver. Replacing the motor would also require better power supply, as a single 18650 battery might not be enough to power it.

Another area of improvement would be the addition of sensors. Adding sensors would allow more precise control of the chambers (e.g. using hall sensors + magnet on each wing to check whether chambers are aligned), as well as tracking whether the pills were dispensed or disposed using an accelerometer at the bottom of dispense chamber. The reason it hasn't been initially done is the lack of documented requirements. On one hand, the project is already quite extensive as it is, on another hand, implementing any of the above mentioned changes would require allocating resources that have already been critically lacking. Doing this would mean revamping a substantial part of the device construction, therefore it is better left for the next iteration of development.

There is also a big area for improvement in the Android App. Due to limited experience and knowledge about current state-of-the-art frontend development paradigm, its usability could further be improved. For this, information like user journey (what functions does user use the most and how long does it take them to get there) need to be collected. Certain functions could also be implemented the other way. For example, bluetooth connection could be done through Android settings and device pairing, removing the need for the device connection screen completely.

References

- [1] C. Haase. “Google i/o 2019: Empowering developers to build the best experiences on android + play.” Blog post, Accessed: May 16, 2025. [Online]. Available: <https://android-developers.googleblog.com/2019/05/google-io-2019-empowering-developers-to-build-experiences-on-Android-Play.html>.
- [2] LiveFine, *Automatic Pill Dispenser with Clear Lid and Large Display*, <https://www.livefineproduct.com/products/automatic-pill-dispenser-with-clear-lid-and-large-display>, Accessed: 2025-04-21, 2024. [Online]. Available: <https://www.livefineproduct.com/products/automatic-pill-dispenser-with-clear-lid-and-large-display>.
- [3] Zoksi. “Zoksi large pill organizer daily pill box for 7 or 14 days wall mountable pill dispenser hold for medicine vitamin or supplements weekly pill case with easy press button blue.” Accessed on April 22, 2025, Accessed: Apr. 22, 2025. [Online]. Available: <https://zoksi.shop/products/zoksi-large-pill-organizer-daily-pill-box-for-7-or-14-days-wall-mountable-pill-dispenser-hold-for-medicine-vitamin-or-supplements-weekly-pill-case-with-easy-press-button-blue>.
- [4] E. Design, *Spur gear generator*, Accessed: 2025-05-02, 2021. [Online]. Available: <https://evolventdesign.com/pages/spur-gear-generator>.
- [5] Kiatronics, *28byj-48 – 5 v 4-phase stepper motor datasheet*, Accessed 2025-05-02, Welten Holdings Ltd., Tauranga, New Zealand. [Online]. Available: <https://www.digikey.com/htmldatasheets/production/1839399/0/0/1/28byj-48.pdf>.
- [6] Texas Instruments Incorporated, “Uln200x, ulq200x high-voltage, high-current darlington transistor arrays,” Texas Instruments Incorporated, Dallas, TX, USA, Datasheet SLRS027T, 2025, Revision T, accessed 2025-05-02. [Online]. Available: <https://www.ti.com/document-viewer/uln2003a/datasheet>.
- [7] Espressif Systems, *Esp32-s3 development kits documentation, User guide v1.1*, Accessed: 2025-05-02, Espressif Systems, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp-dev-kits-en-master-esp32s3.pdf>.
- [8] Panasonic Corporation, “Ncr18650b lithium-ion rechargeable battery cell,” Panasonic Industrial Devices, Datasheet NCR18650B, Accessed 2025-05-02. [Online]. Available: https://batterie18650.pl/wp-content/uploads/2023/12/panasonic_ncr18650b.pdf.
- [9] Prusa Research. “Original Prusa MK4 S 3D Printer.” Accessed 4 May 2025, Accessed: May 4, 2025. [Online]. Available: <https://www.prusa3d.com/product/original-prusa-mk4-2/>.
- [10] S. A. Raj, E. Muthukumaran, and K. Jayakrishna, “A case study of 3d printed pla and its mechanical properties,” *Materials Today: Proceedings*, vol. 5, no. 5, pp. 11 219–11 226, 2018.
- [11] A. Manoj, M. Bhuyan, S. R. Banik, and M. R. Sankar, “Review on particle emissions during fused deposition modeling of acrylonitrile butadiene styrene and polylactic acid polymers,” *Materials Today: Proceedings*, vol. 44, pp. 1375–1383, 2021.

- [12] T. Tábi, A. Wacha, and S. Hajba, “Effect of d-lactide content of annealed poly (lactic acid) on its thermal, mechanical, heat deflection temperature, and creep properties,” *Journal of Applied Polymer Science*, vol. 136, no. 8, p. 47 103, 2019.
- [13] R. Conn et al., “Safety assessment of polylactide (pla) for use as a food-contact polymer,” *Food and Chemical Toxicology*, vol. 33, no. 4, pp. 273–283, 1995.
- [14] Roman Iumatov. “Bachelor thesis source code,” Accessed: May 23, 2025. [Online]. Available: <https://github.com/F3N0mz/Bacharbeit>.
- [15] Bluetooth SIG, “The bluetooth® low energy primer,” Bluetooth Special Interest Group, Tech. Rep., version 1.2.0, Mar. 2024, Accessed: 25 May 2025. [Online]. Available: <https://www.bluetooth.com/wp-content/uploads/2022/05/the-bluetooth-le-primer-v1.2.0.pdf>.
- [16] Institute of Electrical and Electronics Engineers, “Ieee standard for telecommunications and information exchange between systems - lan/man - specific requirements - part 15: Wireless medium access control (mac) and physical layer (phy) specifications for wireless personal area networks (wpans),” *IEEE Std 802.15.1-2002*, pp. 1–473, 2002.
- [17] JetBrains s.r.o. “Basic syntax – variables.” Accessed: May 22, 2025. [Online]. Available: <https://kotlinlang.org/docs/basic-syntax.html#variables>.
- [18] JetBrains s.r.o. “Collections overview – collection types.” Accessed: May 22, 2025. [Online]. Available: <https://kotlinlang.org/docs/collections-overview.html#collection-types>.
- [19] JetBrains s.r.o. “Null safety – kotlin documentation.” Accessed: May 22, 2025. [Online]. Available: <https://kotlinlang.org/docs/null-safety.html>.
- [20] JetBrains s.r.o. “Coroutines, Language support for asynchronous, non-blocking programming.” Accessed: 22 May 2025, Accessed: May 22, 2025. [Online]. Available: <https://kotlinlang.org/docs/coroutines-overview.html>.
- [21] JetBrains s.r.o. “Asynchronous flow, Cold streams that emit multiple values asynchronously.” Accessed: 22 May 2025, Accessed: May 22, 2025. [Online]. Available: <https://kotlinlang.org/docs/flow.html>.
- [22] JetBrains s.r.o. “Stateflow — kotlinx.coroutines,” Accessed: May 23, 2025. [Online]. Available: <https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-state-flow/>.
- [23] JetBrains s.r.o. “Scope functions.” Last modified 25 Sept. 2024, Accessed: May 23, 2025. [Online]. Available: <https://kotlinlang.org/docs/scope-functions.html>.
- [24] Android Developers. “Bluetoothgatt — android api reference,” Google LLC, Accessed: May 23, 2025. [Online]. Available: <https://developer.android.com/reference/kotlin/android/bluetooth/package-summary>.
- [25] Android Developers. “Android.bluetooth.le — android api reference,” Google LLC, Accessed: May 23, 2025. [Online]. Available: <https://developer.android.com/reference/kotlin/android/bluetooth/le/package-summary>.

- [26] R. F. García, “Mvvm: Model–view–viewmodel,” in *iOS Architecture Patterns: MVC, MVP, MVVM, VIPER, and VIP in Swift*. Berkeley, CA: Apress, 2023, pp. 145–224, ISBN: 978-1-4842-9069-9. [Online]. Available: https://doi.org/10.1007/978-1-4842-9069-9_4.
- [27] Android Developers. “Ui layer — app architecture,” Google LLC, Accessed: May 23, 2025. [Online]. Available: <https://developer.android.com/topic/architecture/ui-layer/>.